Gezamenlijke_notebook

March 8, 2026

1 Voorspelling Jeugdpopulatie Almere (2019–2025)

1.1 Gemeente Notebook

Dit notebook bevat het volledige CRISP-DM proces: van probleemdefinitie tot evaluatie.

Voor dit onderzoek gebruiken we historische data van het CBS (Wijken en buurten; kerncijfers). Deze dataset bevat demografische informatie per wijk, zoals het aantal inwoners en het aantal jongeren per leeftijdscategorie.

In dit notebook worden de data eerst verkend en opgeschoond, daarna geanalyseerd en uiteindelijk gebruikt om een voorspellend model te bouwen. Het doel is om inzicht te krijgen in de ontwikkeling van jongeren in Almere en trends zichtbaar te maken die relevant zijn voor toekomstig beleid.

1.2 Business Understanding

Probleem:

De gemeente Almere beschikt over historische gegevens over de bevolkingssamenstelling per wijk, maar heeft nog geen model dat toekomstige ontwikkelingen van de jeugdpopulatie kan voorspellen. Hierdoor ontbreekt een datagedreven onderbouwing voor het plannen van voorzieningen zoals scholen, jongerenvoorzieningen en jeugdzorg.

Zonder inzicht in toekomstige ontwikkelingen bestaat het risico dat voorzieningen niet goed aansluiten op de vraag in verschillende wijken.

Doel:

De ontwikkeling van de jeugdpopulatie heeft directe invloed op voorzieningen in de stad. Wanneer het aantal jongeren in een wijk groeit, kan dit leiden tot een grotere behoefte aan scholen, sportvoorzieningen en jeugdzorg.

Door trends in de jeugdpopulatie tijdig te signaleren kan de gemeente Almere beter plannen en middelen efficiënter inzetten. Datagedreven inzichten helpen beleidsmakers om toekomstige ontwikkelingen beter te onderbouwen en voorbereid te zijn op veranderingen in de stad.

Waarom is dit belangrijk?

De ontwikkeling van de jeugdpopulatie heeft directe invloed op voorzieningen in de stad. Wanneer het aantal jongeren in een wijk groeit, kan dit leiden tot een grotere behoefte aan scholen, sportvoorzieningen en jeugdzorg.

Door trends in de jeugdpopulatie tijdig te signaleren kan de gemeente Almere beter plannen en middelen efficiënter inzetten. Datagedreven inzichten helpen beleidsmakers om toekomstige ontwikkelingen beter te onderbouwen en voorbereid te zijn op veranderingen in de stad.

---

2 Data Preparation – Fase 1

In deze eerste fase van de data preparation wordt de ruwe CBS-dataset ingelezen en verkend. Het doel van deze stap is om inzicht te krijgen in de structuur en kwaliteit van de data voordat verdere analyses worden uitgevoerd.

We beginnen met een eerste verkenning van de dataset. Hierbij kijken we naar het aantal rijen en kolommen, de beschikbare variabelen, de datatypes en mogelijke ontbrekende waarden. Deze stap helpt om te begrijpen hoe de data is opgebouwd en welke voorbereidingen nodig zijn voor verdere analyse.

De dataset bevat demografische gegevens per regio, waaronder het aantal inwoners en de verdeling van inwoners over verschillende leeftijdscategorieën. Omdat dit onderzoek zich specifiek richt op de jeugdpopulatie in Almere, zullen we in de volgende stappen de dataset filteren op deze gemeente en op wijkniveau.

Het doel van deze fase is om een opgeschoonde dataset te verkrijgen die geschikt is voor verdere analyse in de volgende stappen van het project.

2.0.1 Inladen van de dataset

In deze stap importeren we de benodigde Python-libraries voor data-analyse en visualisatie. Vervolgens wordt de ruwe CBS-dataset ingeladen. Met `shape` bekijken we de grootte van de dataset en met `head()` krijgen we een eerste indruk van de structuur en inhoud van de data.

[1]:
```
# importeren van benodigde libraries
import pandas as pd
from pathlib import Path
import matplotlib.pyplot as plt
import seaborn as sns

# Ruwe CBS dataset inladen
df_cbs = pd.read_csv("../../data/raw/CBS_Kerncijfers.csv", sep=";")

print("Shape:", df_cbs.shape)
df_cbs.head()
```

```
Shape: (1536, 411)
```

[1]:
```
     ID Peildatum WijkenEnBuurten                                Gemeentenaam_1  \
0  384  1-1-2025      GM0034      Almere
1  385  1-1-2025      WK003401    Almere
2  386  1-1-2025      BU00340101  Almere
3  387  1-1-2025      BU00340102  Almere
```

```
4  388  1-1-2025      WK003402    Almere

  SoortRegio_2  Codering_3  IndelingswijzigingGemeenteWijkBuurt_4  \
0   Gemeente    GM0034                                       1.0
1   Wijk        WK003401                                     1.0
2   Buurt       BU00340101                                   1.0
3   Buurt       BU00340102                                   1.0
4   Wijk        WK003402                                     1.0

   AantalInwoners_5  Mannen_6  Vrouwen_7  …  PersonenPerSoortUitkeringAO_81  \
0            229574    113738     115836  …                              NaN
1              1810       880        930  …                              NaN
2               795       415        385  …                              NaN
3              1010       465        545  …                              NaN
4              2890      1395       1500  …                              NaN

   PersonenPerSoortUitkeringWW_82  PersonenPerSoortUitkeringAOW_83  \
0                             NaN                              NaN
1                             NaN                              NaN
2                             NaN                              NaN
3                             NaN                              NaN
4                             NaN                              NaN

   JongerenMetJeugdzorgInNatura_84  PercentageJongerenMetJeugdzorg_85  \
0                              NaN                                NaN
1                              NaN                                NaN
2                              NaN                                NaN
3                              NaN                                NaN
4                              NaN                                NaN

   WmoClienten_86  WmoClientenRelatief_87  TotaalDiefstalUitWoningSchuurED_88  \
0             NaN                     NaN                                 NaN
1             NaN                     NaN                                 NaN
2             NaN                     NaN                                 NaN
3             NaN                     NaN                                 NaN
4             NaN                     NaN                                 NaN

   VernielingMisdrijfTegenOpenbareOrde_89  GeweldsEnSeksueleMisdrijven_90
0                                     NaN                             NaN
1                                     NaN                             NaN
2                                     NaN                             NaN
3                                     NaN                             NaN
4                                     NaN                             NaN

[5 rows x 411 columns]
```

Eerste indruk van de dataset

De eerste rijen van de dataset laten zien dat de data bevolkingsinformatie bevat per gemeente, wijk en buurt. Voor dit onderzoek zijn de rijen op wijkniveau relevant, omdat we verschillen tussen wijken in Almere willen analyseren.

De dataset bevat veel variabelen (411 kolommen), waaronder het aantal inwoners, geslacht, leeftijdsgroepen en verschillende sociaal-economische indicatoren. In de volgende stappen wordt de dataset gefilterd op gemeente Almere en op wijkniveau, zodat alleen de relevante gegevens overblijven.

2.1 Data Preparation FASE 1

2.1.1 Overzicht van de dataset

Eerste verkenning (Wail + Faysal)

In deze stap verkennen we de structuur van de dataset. We bekijken de kolommen, datatypes en basisstatistieken om een eerste beeld te krijgen van de beschikbare gegevens. Daarnaast analyseren we het aantal missende waarden per kolom en het percentage ontbrekende data.

[2]:
```
# Overzicht: kolommen, types, missende waarden
df_cbs.info()

# Beschrijvende statistiek (numeriek)
df_cbs.describe()
# Missende waarden overzicht (aantallen + percentages)
missing_counts = df_cbs.isnull().sum()
missing_pct = (missing_counts / len(df_cbs)) * 100

missing_overview = pd.DataFrame({
    "missing_count": missing_counts,
    "missing_percentage": missing_pct
}).sort_values("missing_percentage", ascending=False)

missing_overview
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 1536 entries, 0 to 1535
Columns: 411 entries, ID to GeweldsEnSeksueleMisdrijven_90
dtypes: float64(387), int64(13), object(11)
memory usage: 4.8+ MB
```

[2]:

	missing_count	missing_percentage
HJVervoerInformatieEnCommunicatie_101	1536	100.0
GemiddeldeElektriciteitsleveringTotaal_53	1536	100.0
PersonenPerSoortUitkeringBijstand_89	1536	100.0
PersonenPerSoortUitkeringAO_90	1536	100.0
PersonenPerSoortUitkeringWW_91	1536	100.0
…	…	…
Gehuwd_14	0	0.0
Gescheiden_15	0	0.0
Verweduwd_16	0	0.0

```
Peildatum                                                  0                 0.0
ID                                                         0                 0.0

[411 rows x 2 columns]
```

Toelichting

Uit de analyse blijkt dat de dataset een groot aantal variabelen bevat en dat sommige kolommen veel missende waarden hebben. Dit is deels te verklaren doordat de dataset verschillende soorten regio's bevat, zoals gemeenten, wijken en buurten, waarbij niet alle indicatoren voor elk niveau beschikbaar zijn.

Door dit overzicht krijgen we inzicht in de kwaliteit van de data en kunnen we bepalen welke variabelen relevant zijn voor het verdere onderzoek en welke eventueel moeten worden opgeschoond of verwijderd.

2.1.2 Filter op Almere en op wijken (Ismael)

In deze stap wordt de dataset opgeschoond en gefilterd zodat alleen de relevante gegevens voor dit onderzoek overblijven. Eerst worden kolomnamen en tekstvelden opgeschoond om eventuele spaties te verwijderen. Vervolgens wordt de dataset gefilterd op de gemeente Almere. Daarna wordt verder gefilterd op regio's van het type wijk, omdat dit onderzoek zich richt op verschillen tussen wijken binnen Almere.

[3]:
```python
# kolomnamen opschonen
df_cbs.columns = df_cbs.columns.astype(str).str.strip()

# gemeentenaam opschonen
df_cbs["Gemeentenaam_1"] = df_cbs["Gemeentenaam_1"].astype(str).str.strip()

# filter Almere
df_almere = df_cbs[df_cbs["Gemeentenaam_1"] == "Almere"].copy()
print("Shape Almere:", df_almere.shape)

# waardes opschonen + filter wijken
df_almere["SoortRegio_2"] = df_almere["SoortRegio_2"].astype(str).str.strip()
df_almere["Codering_3"] = df_almere["Codering_3"].astype(str).str.strip()

df_wijk = df_almere[df_almere["SoortRegio_2"] == "Wijk"].copy()
df_wijk = df_wijk[df_wijk["Codering_3"].str.startswith("WK")].copy()

print("Shape wijken:", df_wijk.shape)
df_wijk.head()
```

```
Shape Almere: (1536, 411)
Shape wijken: (308, 411)
```

[3]:
```
     ID Peildatum WijkenEnBuurten Gemeentenaam_1 SoortRegio_2 Codering_3  \
1   385  1-1-2025      WK003401           Almere         Wijk   WK003401
4   388  1-1-2025      WK003402           Almere         Wijk   WK003402
```

```
10  394  1-1-2025      WK003403           Almere         Wijk   WK003403
15  399  1-1-2025      WK003404           Almere         Wijk   WK003404
19  403  1-1-2025      WK003405           Almere         Wijk   WK003405

    IndelingswijzigingGemeenteWijkBuurt_4  AantalInwoners_5  Mannen_6  \
1                                     1.0              1810       880
4                                     1.0              2890      1395
10                                    1.0              2500      1205
15                                    1.0              2220      1065
19                                    1.0              2365      1130

    Vrouwen_7  …  PersonenPerSoortUitkeringAO_81  \
1         930  …                             NaN
4        1500  …                             NaN
10       1300  …                             NaN
15       1155  …                             NaN
19       1235  …                             NaN

    PersonenPerSoortUitkeringWW_82  PersonenPerSoortUitkeringAOW_83  \
1                              NaN                               NaN
4                              NaN                               NaN
10                             NaN                               NaN
15                             NaN                               NaN
19                             NaN                               NaN

    JongerenMetJeugdzorgInNatura_84  PercentageJongerenMetJeugdzorg_85  \
1                               NaN                                NaN
4                               NaN                                NaN
10                              NaN                                NaN
15                              NaN                                NaN
19                              NaN                                NaN

    WmoClienten_86  WmoClientenRelatief_87  \
1              NaN                      NaN
4              NaN                      NaN
10             NaN                      NaN
15             NaN                      NaN
19             NaN                      NaN

    TotaalDiefstalUitWoningSchuurED_88  \
1                                  NaN
4                                  NaN
10                                 NaN
15                                 NaN
19                                 NaN

    VernielingMisdrijfTegenOpenbareOrde_89  GeweldsEnSeksueleMisdrijven_90
```

```
1                                       NaN                                 NaN
4                                       NaN                                 NaN
10                                      NaN                                 NaN
15                                      NaN                                 NaN
19                                      NaN                                 NaN

[5 rows x 411 columns]
```

Toelichting

Na het filteren bevat de dataset 1536 rijen voor de gemeente Almere. Wanneer vervolgens alleen de regio's van het type wijk worden geselecteerd, blijven er 308 rijen over. Deze rijen bevatten de demografische gegevens per wijk en per jaar. Deze gefilterde dataset vormt de basis voor de verdere analyses in het onderzoek.

2.1.3 Basis datakwaliteit checks (Wail + Noah + Ismael)

In deze stap controleren we de kwaliteit en volledigheid van de wijkdata. We kijken of er dubbele rijen of dubbele combinaties van wijkcode en peildatum voorkomen. Daarnaast controleren we hoeveel unieke wijken er in de dataset zitten en of voor elke wijk alle jaren aanwezig zijn.

[4]:
```python
# Dubbele combinaties wijkcode + peildatum
dup_count = df_wijk.duplicated(subset=["Codering_3", "Peildatum"]).sum()
print("Duplicaten (Codering_3 + Peildatum):", dup_count)

# Volledig dubbele rijen
full_dup = df_wijk.duplicated().sum()
print("Volledig dubbele rijen:", full_dup)

# Unieke wijken
print("Aantal unieke wijken:", df_wijk["Codering_3"].nunique())

# Aantal jaren per wijk
jaren_per_wijk = df_wijk.groupby("Codering_3")["Peildatum"].nunique()
jaren_per_wijk.describe()

# Missende jaren per wijk
alle_jaren = sorted(df_wijk["Peildatum"].dropna().unique())

print("Missende jaren per wijk (alleen wijken met missende jaren):")
for wijk, groep in df_wijk.groupby("Codering_3"):
    aanwezige = sorted(groep["Peildatum"].dropna().unique())
    missend = sorted(set(alle_jaren) - set(aanwezige))
    if missend:
        print(f"{wijk} mist: {missend}")
```

```
Duplicaten (Codering_3 + Peildatum): 0
Volledig dubbele rijen: 0
Aantal unieke wijken: 71
```

```
Missende jaren per wijk (alleen wijken met missende jaren):
WK003407 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003408 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003409 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003410 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003411 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003414 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003415 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003416 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003417 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003418 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003419 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003420 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003421 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003422 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003423 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003424 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003425 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003426 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003427 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003428 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003429 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003430 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003431 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003432 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003434 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003435 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003436 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003437 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003438 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003439 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003440 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003441 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003442 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003443 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003444 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003445 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003446 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003447 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003448 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003449 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003450 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003451 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003452 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003453 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003456 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003457 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003458 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
```

```
WK003459 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003460 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003461 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003462 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003463 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003464 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003465 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003466 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003471 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003472 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003473 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003474 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003475 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003476 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003477 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003479 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003485 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
WK003496 mist: ['1-1-2018', '1-1-2019', '1-1-2020', '1-1-2021']
```

Toelichting

Uit de controle blijkt dat er geen dubbele combinaties van wijkcode en peildatum zijn en ook geen volledig dubbele rijen. Dat betekent dat de dataset op dit punt consistent is. Verder bevat de dataset 71 unieke wijken.

Wel blijkt dat meerdere wijken de jaren 2018 tot en met 2021 missen. Dit wijst erop dat niet alle wijken in alle jaren voorkomen, waarschijnlijk door wijzigingen in de wijkindeling of doordat sommige wijken pas later zijn toegevoegd. Dit is belangrijk om mee te nemen in de verdere analyse en modellering.

2.1.4 Kernvariabelen opschonen (Bo + Faysal)

In deze stap worden de belangrijkste variabelen voor het onderzoek opgeschoond. De kolommen voor het totaal aantal inwoners, het aantal inwoners van 0–15 jaar en het aantal inwoners van 15–25 jaar worden omgezet naar numerieke waarden. Daarna worden rijen met ontbrekende waarden en nulwaarden verwijderd, zodat alleen bruikbare observaties overblijven voor de verdere analyse.

[5]:
```python
cols_check = ["AantalInwoners_5", "k_0Tot15Jaar_8", "k_15Tot25Jaar_9"]

# naar numeriek
for col in cols_check:
    df_wijk[col] = pd.to_numeric(df_wijk[col], errors="coerce")

# NaN eruit
df_wijk = df_wijk.dropna(subset=cols_check)

# 0-waarden eruit
for col in cols_check:
    df_wijk = df_wijk[df_wijk[col] != 0]
```

```
print("Shape na opschonen kernvariabelen:", df_wijk.shape)
df_wijk[cols_check].describe()

# Extra logische checks (Faysel)
kol_totaal = "AantalInwoners_5"
kol_0_15 = "k_0Tot15Jaar_8"
kol_15_25 = "k_15Tot25Jaar_9"

checks = {
    "negatief_0_15": (df_wijk[kol_0_15] < 0).sum(),
    "negatief_15_25": (df_wijk[kol_15_25] < 0).sum(),
    "negatief_totaal": (df_wijk[kol_totaal] < 0).sum(),
    "0_15_groter_dan_totaal": (df_wijk[kol_0_15] > df_wijk[kol_totaal]).sum(),
    "15_25_groter_dan_totaal": (df_wijk[kol_15_25] > df_wijk[kol_totaal]).sum(),
}
checks
```

```
Shape na opschonen kernvariabelen: (237, 411)
```

[5]:
```
{'negatief_0_15': 0,
 'negatief_15_25': 0,
 'negatief_totaal': 0,
 '0_15_groter_dan_totaal': 0,
 '15_25_groter_dan_totaal': 0}
```

Toelichting

Na het opschonen blijven er 237 rijen in de dataset over. Daarnaast laten de logische controles zien dat er geen negatieve waarden voorkomen en dat het aantal jongeren nooit groter is dan het totaal aantal inwoners. Dit betekent dat de kernvariabelen consistent en geschikt zijn voor verdere analyse.

2.1.5 Kolommen duidelijk maken + tijdstructuur (Ismael + Mo + Wail)

2.1.6 Structureren van de dataset

In deze stap worden de kolomnamen aangepast naar duidelijkere en beter leesbare namen. Vervolgens wordt de variabele Peildatum omgezet naar een datumformaat en wordt hieruit een nieuwe variabele Jaar gemaakt. De dataset wordt daarna gesorteerd op wijk en jaar zodat de gegevens geschikt zijn voor tijdreeksanalyse. Tot slot worden alleen de relevante kolommen geselecteerd voor het verdere onderzoek.

[6]:
```
# Kolommen hernoemen naar duidelijke namen
df_wijk = df_wijk.rename(columns={
    "WijkenEnBuurten": "Wijk",
    "Codering_3": "WijkCode",
    "AantalInwoners_5": "AantalInwoners",
    "k_0Tot15Jaar_8": "Aantal_0_15",
```

```python
    "k_15Tot25Jaar_9": "Aantal_15_25",
})

# Peildatum omzetten naar datetime + Jaar maken
df_wijk["Peildatum"] = pd.to_datetime(df_wijk["Peildatum"], errors="coerce")
df_wijk["Jaar"] = df_wijk["Peildatum"].dt.year

# erwijder rijen zonder geldige datum/jaar
df_wijk = df_wijk.dropna(subset=["Peildatum", "Jaar"]).copy()
df_wijk["Jaar"] = df_wijk["Jaar"].astype(int)

# sorteren (handig voor tijdreeksen)
df_wijk = df_wijk.sort_values(["WijkCode", "Jaar"]).reset_index(drop=True)

# Duplicaten check (wijkcode + jaar)
dup_wijk_jaar = df_wijk.duplicated(subset=["WijkCode", "Jaar"]).sum()
print("Duplicaten (WijkCode + Jaar):", dup_wijk_jaar)

# Kolommen selecteren (Wail)
keep_cols = [
    "Peildatum", "Jaar", "WijkCode", "Wijk",
    "AantalInwoners", "Aantal_0_15", "Aantal_15_25"
]

missing = [c for c in keep_cols if c not in df_wijk.columns]
if missing:
    print("Ontbrekende kolommen:", missing)

df_wijk = df_wijk[[c for c in keep_cols if c in df_wijk.columns]].copy()

print("Kolommen na selectie:", df_wijk.columns.tolist())
print("Shape na selectie:", df_wijk.shape)

df_wijk.head()
```

```
Duplicaten (WijkCode + Jaar): 0
Kolommen na selectie: ['Peildatum', 'Jaar', 'WijkCode', 'Wijk',
'AantalInwoners', 'Aantal_0_15', 'Aantal_15_25']
Shape na selectie: (237, 7)
```

[6]:

	Peildatum	Jaar	WijkCode	Wijk	AantalInwoners	Aantal_0_15	\
0	2018-01-01	2018	WK003401	WK003401	22790	3975	
1	2019-01-01	2019	WK003401	WK003401	22875	3945	
2	2020-01-01	2020	WK003401	WK003401	23275	4005	
3	2021-01-01	2021	WK003401	WK003401	23530	4180	
4	2022-01-01	2022	WK003401	WK003401	1635	175	

```
   Aantal_15_25
0          2420
1          2415
2          2485
3          2465
4           150
```

Toelichting

Na deze stap bevat de dataset 237 rijen en 7 kolommen met de belangrijkste variabelen voor het onderzoek. De dataset bevat nu per wijk en per jaar het totaal aantal inwoners en het aantal inwoners in de leeftijdscategorieën 0–15 jaar en 15–25 jaar. Er zijn geen dubbele combinaties van wijkcode en jaar gevonden, wat betekent dat elke wijk per jaar slechts één keer voorkomt. Hierdoor is de dataset overzichtelijk en geschikt voor verdere analyse.

2.1.7 Percentages toevoegen (Faysel) – handig voor latere analyse

In deze stap worden de kolomnamen aangepast naar duidelijkere en beter leesbare namen. Daarnaast wordt de variabele Peildatum omgezet naar een datumformaat en wordt hieruit een nieuwe variabele Jaar gemaakt. De dataset wordt vervolgens gesorteerd op wijk en jaar, zodat de gegevens geschikt zijn voor tijdreeksanalyse. Tot slot worden alleen de relevante kolommen geselecteerd voor het verdere onderzoek.

[7]:
```
df_wijk["pct_0_15"] = (df_wijk["Aantal_0_15"] / df_wijk["AantalInwoners"]) * 100
df_wijk["pct_15_25"] = (df_wijk["Aantal_15_25"] / df_wijk["AantalInwoners"]) *␣
↪100

df_wijk[["pct_0_15", "pct_15_25"]].describe()
```

[7]:
```
         pct_0_15   pct_15_25
count  237.000000  237.000000
mean    18.277507   12.805460
std      6.034239    6.198061
min      2.083333    6.250000
25%     15.671642   10.135135
50%     17.287234   11.790393
75%     19.288646   14.258735
max     57.142857   83.333333
```

Toelichting

Na deze stap bevat de dataset 237 rijen en 7 kolommen met de belangrijkste variabelen voor het onderzoek. De dataset bevat nu per wijk en per jaar het totaal aantal inwoners en het aantal inwoners in de leeftijdscategorieën 0–15 jaar en 15–25 jaar. Er zijn geen dubbele combinaties van wijkcode en jaar gevonden, wat betekent dat elke wijk per jaar slechts één keer voorkomt in de dataset. Hierdoor is de dataset geschikt voor verdere analyse en modellering.

2.1.8 Export: dataset klaar voor analyse

In deze stap wordt de opgeschoonde en gestructureerde dataset opgeslagen als een nieuwe CSV-bestand. Hierdoor kan de dataset later eenvoudig opnieuw worden ingeladen voor verdere analyse en modellering.

[8]:
```
output_path = Path("../../data/processed/df_v8_ready_for_analysis.csv")
df_wijk.to_csv(output_path, index=False)

output_path.as_posix()
```

[8]:
```
'../../data/processed/df_v8_ready_for_analysis.csv'
```

Toelichting

De dataset wordt opgeslagen onder de naam df_v8_ready_for_analysis.csv in de map data/processed. Deze dataset bevat alleen de relevante variabelen en een consistente structuur per wijk en per jaar. Dit bestand vormt de basis voor de volgende fase van het project, waarin de data verder wordt geanalyseerd.

2.2 Data Understanding FASE 2

In deze fase wordt de opgeschoonde dataset verder verkend om inzicht te krijgen in patronen, trends en relaties tussen de variabelen. Door middel van visualisaties en statistische analyses onderzoeken we hoe de jeugdpopulatie zich ontwikkelt over de jaren en hoe deze verschilt tussen wijken. Deze verkenning helpt om de data beter te begrijpen en vormt de basis voor de verdere modellering.

2.2.1 Inladen van de analyseklare dataset

In deze stap laden we de opgeschoonde dataset die in fase 1 is gemaakt. Deze dataset bevat alleen de relevante variabelen per wijk en per jaar en vormt de basis voor de verdere verkennende analyse.

[9]:
```
df = pd.read_csv("../../data/processed/df_v8_ready_for_analysis.csv")
print("Shape:", df.shape)
df.head()
```

```
Shape: (237, 9)
```

[9]:
```
     Peildatum  Jaar  WijkCode        Wijk  AantalInwoners  Aantal_0_15  \
0  2018-01-01  2018  WK003401  WK003401             22790         3975
1  2019-01-01  2019  WK003401  WK003401             22875         3945
2  2020-01-01  2020  WK003401  WK003401             23275         4005
3  2021-01-01  2021  WK003401  WK003401             23530         4180
4  2022-01-01  2022  WK003401  WK003401              1635          175

   Aantal_15_25   pct_0_15  pct_15_25
0          2420  17.441860  10.618692
1          2415  17.245902  10.557377
2          2485  17.207304  10.676692
```

```
3          2465  17.764556  10.475988
4           150  10.703364   9.174312
```

Toelichting

De dataset bevat 237 rijen en 9 kolommen. Voor elke wijk en elk jaar zijn het totaal aantal inwoners, het aantal inwoners van 0–15 jaar en het aantal inwoners van 15–25 jaar beschikbaar. Daarnaast zijn ook de percentages van deze leeftijdsgroepen ten opzichte van de totale bevolking toegevoegd. Deze dataset wordt gebruikt om trends en relaties in de jeugdpopulatie verder te analyseren.

2.2.2 Voorbereiden van de tijdstructuur

Tijdreeks voorbereiden (Mo + Ismael)

In deze stap wordt de variabele Peildatum omgezet naar een datumformaat en wordt hieruit het jaar gehaald. Daarna wordt de dataset gesorteerd op wijk en jaar, zodat de gegevens in chronologische volgorde staan.

[10]:
```
df["Peildatum"] = pd.to_datetime(df["Peildatum"], errors="coerce")
df["Jaar"] = df["Peildatum"].dt.year

df = df.sort_values(["Wijk", "Jaar"]).reset_index(drop=True)

df[["Wijk","Jaar"]].head()
```

[10]:
```
         Wijk  Jaar
0  WK003401    2018
1  WK003401    2019
2  WK003401    2020
3  WK003401    2021
4  WK003401    2022
```

Toelichting

Na deze stap is de dataset gestructureerd als een tijdreeks per wijk. Dit betekent dat de gegevens per wijk netjes op volgorde van jaar staan, wat belangrijk is voor het analyseren van trends en ontwikkelingen over de tijd.

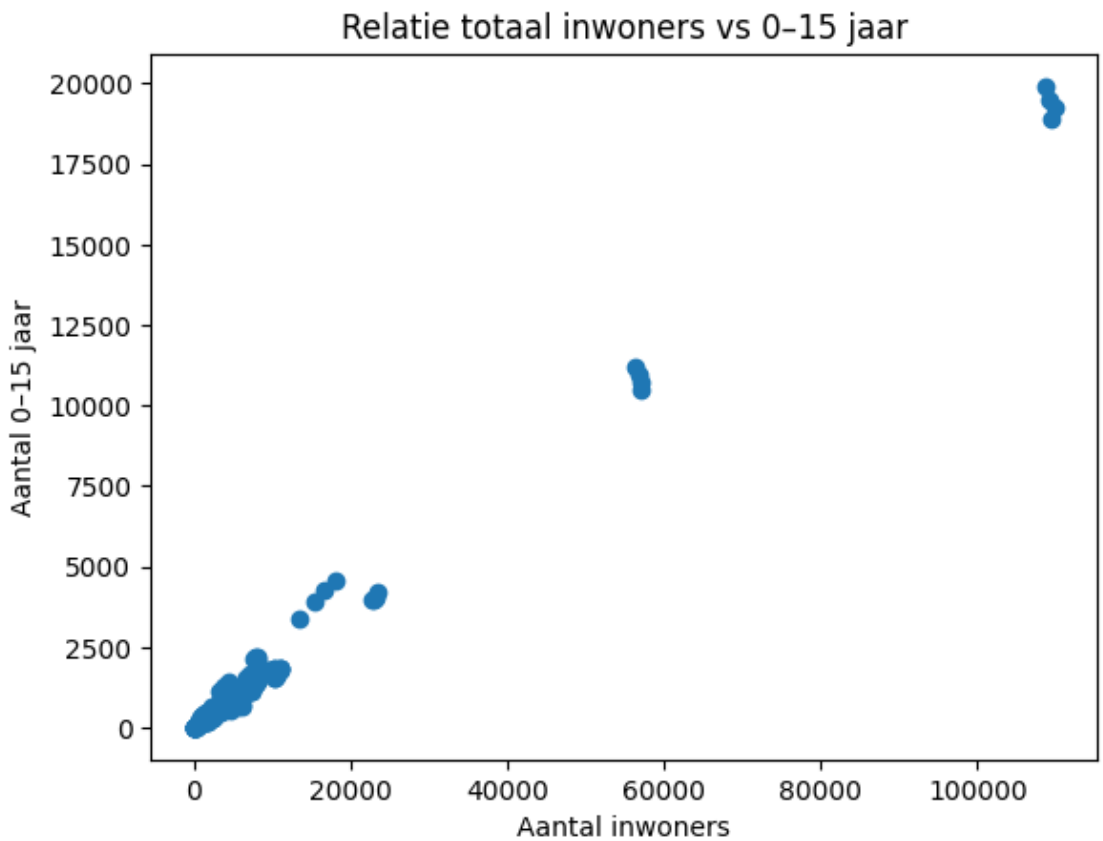
2.2.3 Relatie tussen totaal aantal inwoners en jongeren

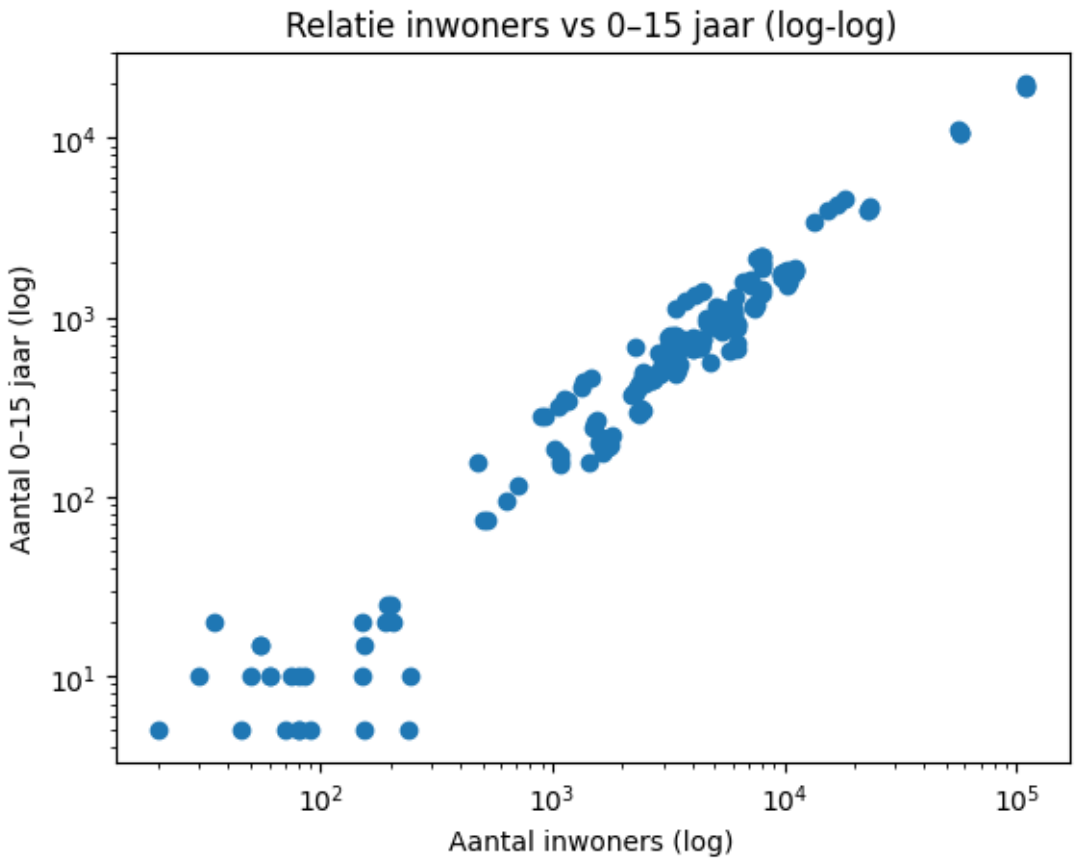
Scatterplot relatie inwoners vs jongeren (Mo + Faysel)

In deze stap onderzoeken we de relatie tussen het totaal aantal inwoners en het aantal kinderen van 0–15 jaar per wijk. Hiervoor maken we een scatterplot. Daarnaast wordt dezelfde relatie ook weergegeven op een log-log schaal om patronen beter zichtbaar te maken wanneer de waarden sterk verschillen in grootte.

[11]:
```
plt.scatter(df["AantalInwoners"], df["Aantal_0_15"])
plt.xlabel("Aantal inwoners")
plt.ylabel("Aantal 0-15 jaar")
plt.title("Relatie totaal inwoners vs 0-15 jaar")
```

```
plt.show()


plt.scatter(df["AantalInwoners"], df["Aantal_0_15"])
plt.xscale("log")
plt.yscale("log")
plt.xlabel("Aantal inwoners (log)")
plt.ylabel("Aantal 0-15 jaar (log)")
plt.title("Relatie inwoners vs 0-15 jaar (log-log)")
plt.show()
```





Toelichting

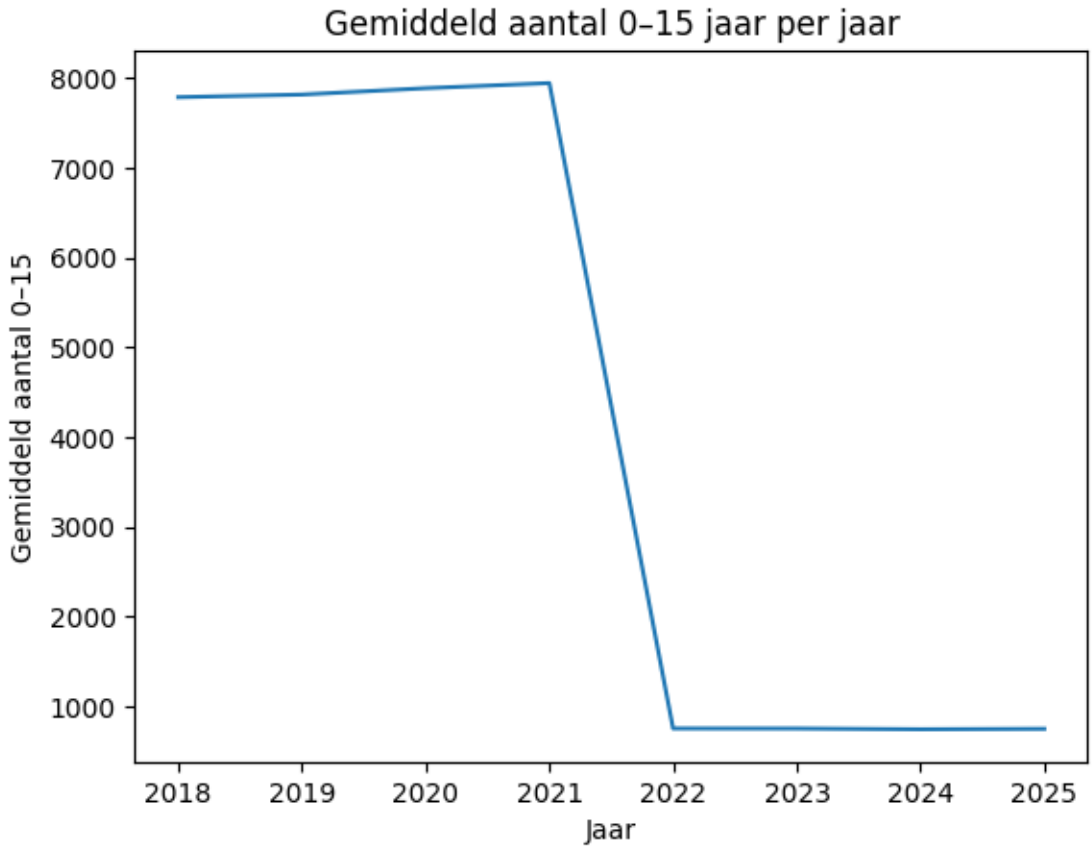
De grafieken laten een duidelijke positieve relatie zien tussen het totaal aantal inwoners en het aantal kinderen van 0–15 jaar. Wijken met meer inwoners hebben over het algemeen ook meer kinderen. In de log-log grafiek wordt deze relatie nog duidelijker zichtbaar, omdat grote en kleine wijken beter met elkaar vergeleken kunnen worden. Dit wijst erop dat de omvang van de bevolking een belangrijke factor is voor het aantal jongeren in een wijk.

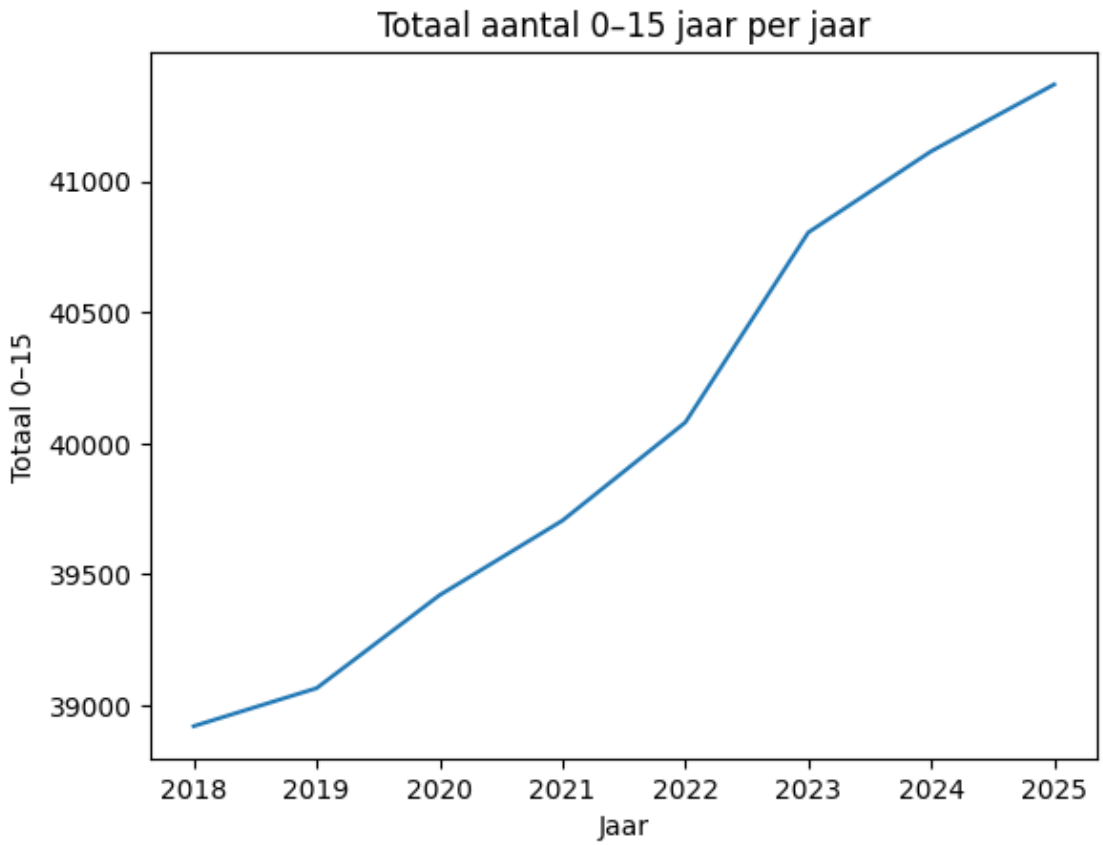
2.2.4 Trend van jongeren door de jaren (Mohammed)

In deze stap onderzoeken we hoe het aantal kinderen van 0–15 jaar zich ontwikkelt over de tijd. Hiervoor bekijken we zowel het gemiddelde aantal kinderen per wijk per jaar als het totale aantal kinderen in Almere per jaar.

[12]:
```
df.groupby("Jaar")["Aantal_0_15"].mean().plot()
plt.title("Gemiddeld aantal 0-15 jaar per jaar")
plt.xlabel("Jaar")
plt.ylabel("Gemiddeld aantal 0-15")
plt.show()
```

```
df.groupby("Jaar")["Aantal_0_15"].sum().plot()
plt.title("Totaal aantal 0-15 jaar per jaar")
plt.xlabel("Jaar")
plt.ylabel("Totaal 0-15")
plt.show()
```





Toelichting

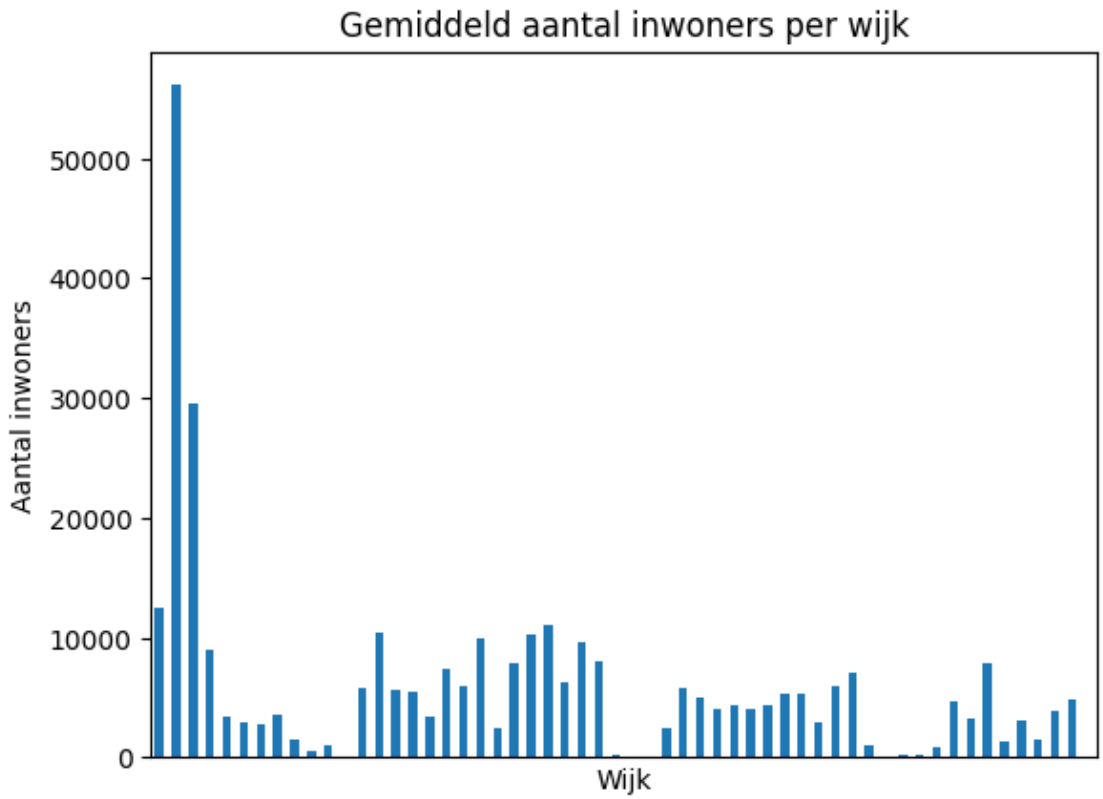
De grafieken laten zien dat het totale aantal kinderen van 0–15 jaar in Almere geleidelijk toeneemt over de jaren. Dit wijst op een groei van de jeugdpopulatie in de stad. In de grafiek met het gemiddelde per wijk is een sterke daling zichtbaar vanaf 2022. Dit komt waarschijnlijk doordat vanaf dat jaar meer wijken in de dataset zijn opgenomen, waardoor het gemiddelde per wijk lager uitvalt. Daarom is het totaal aantal kinderen per jaar een betrouwbaardere indicator voor de algemene trend in de jeugdpopulatie.

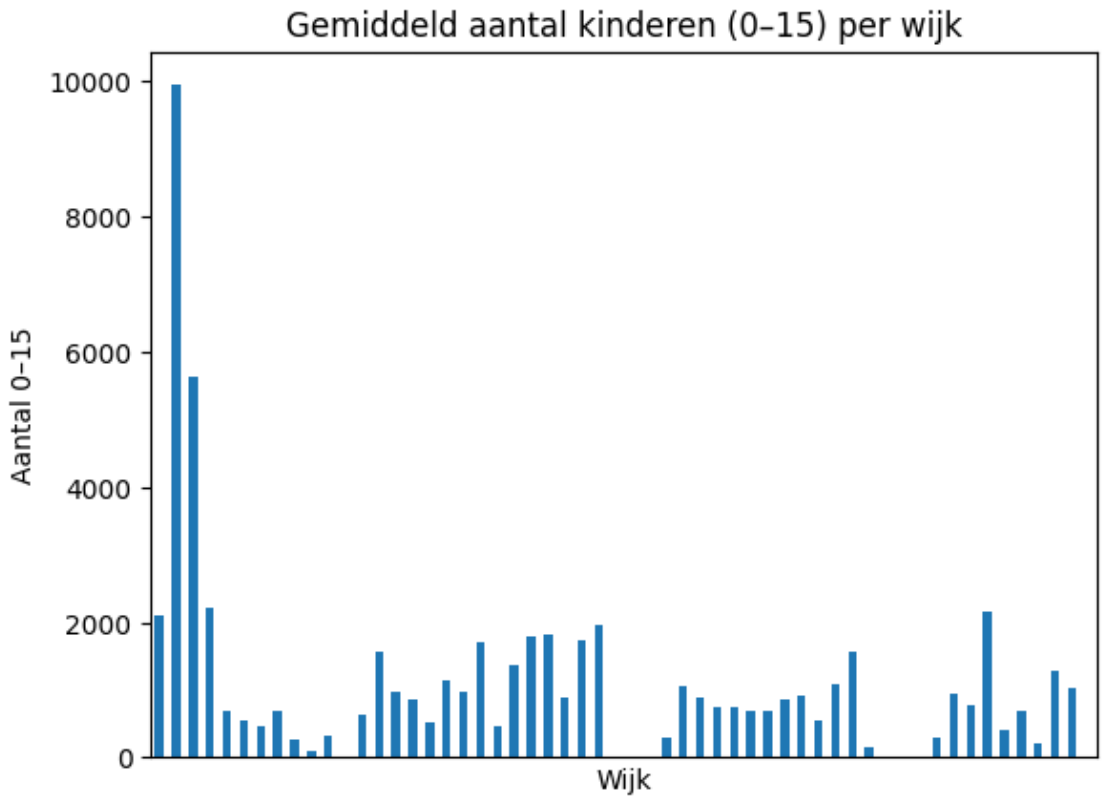
2.2.5 Vergelijking tussen wijken (Faysel)

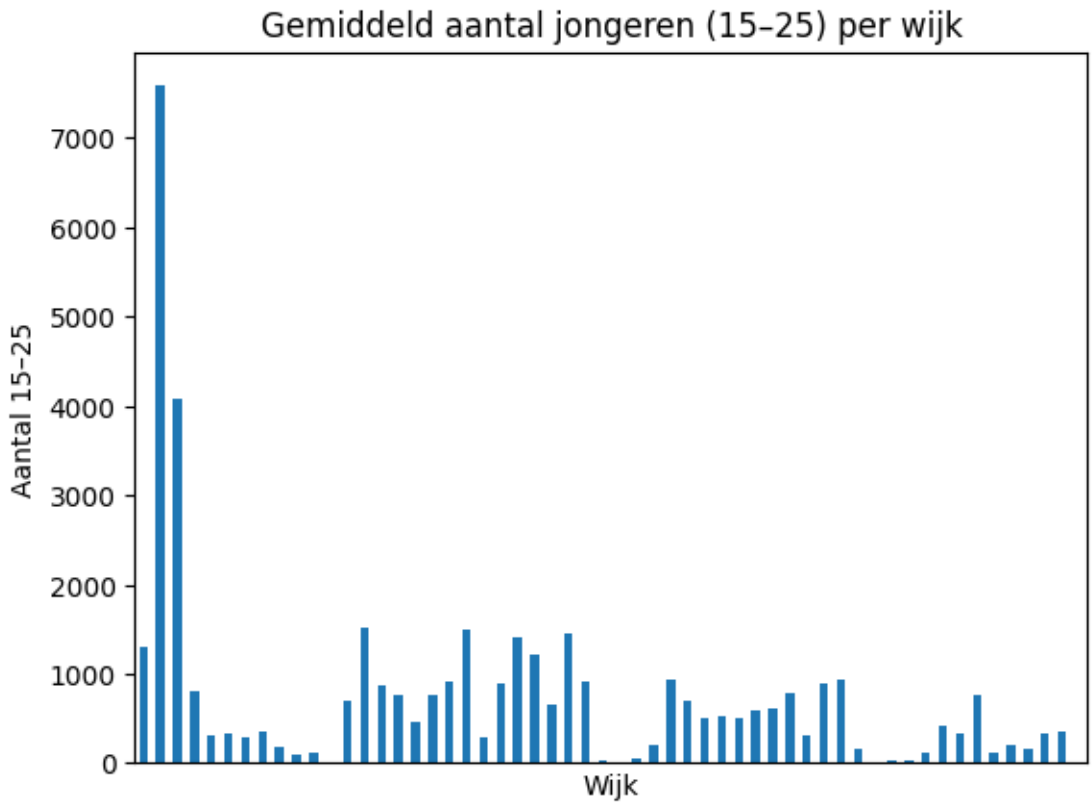
In deze stap vergelijken we het gemiddelde aantal inwoners, kinderen (0–15 jaar) en jongeren (15–25 jaar) per wijk. Door deze grafieken te bekijken krijgen we inzicht in de verschillen tussen wijken in Almere.

[13]:
```
df.groupby("Wijk")["AantalInwoners"].mean().plot(kind="bar")
plt.title("Gemiddeld aantal inwoners per wijk")
plt.ylabel("Aantal inwoners")
plt.xticks([])
plt.show()
```

```
df.groupby("Wijk")["Aantal_0_15"].mean().plot(kind="bar")
plt.title("Gemiddeld aantal kinderen (0-15) per wijk")
plt.ylabel("Aantal 0-15")
plt.xticks([])
plt.show()

df.groupby("Wijk")["Aantal_15_25"].mean().plot(kind="bar")
plt.title("Gemiddeld aantal jongeren (15-25) per wijk")
plt.ylabel("Aantal 15-25")
plt.xticks([])
plt.show()
```




Gemiddeld aantal kinderen (0–15) per wijk
Aantal 0–15
10000
8000
6000
4000
2000
0
Wijk



Toelichting

De grafieken laten zien dat er grote verschillen bestaan tussen wijken in Almere. Sommige wijken hebben een veel grotere bevolking en daardoor ook meer kinderen en jongeren. Tegelijkertijd zijn er ook wijken met relatief weinig inwoners en een kleinere jeugdpopulatie.

De patronen in de drie grafieken lijken sterk op elkaar. Wijken met veel inwoners hebben meestal ook meer kinderen en jongeren. Dit bevestigt dat de omvang van de bevolking een belangrijke factor is bij het verklaren van het aantal jongeren in een wijk.

2.2.6 Correlaties tussen variabelen (Faysal)

In deze stap onderzoeken we de correlatie tussen het totaal aantal inwoners, het aantal kinderen van 0–15 jaar en het aantal jongeren van 15–25 jaar. Correlatie geeft aan hoe sterk twee variabelen samenhangen.

[14]:
```
df[["AantalInwoners", "Aantal_0_15", "Aantal_15_25"]].corr()
```

[14]:

	AantalInwoners	Aantal_0_15	Aantal_15_25
AantalInwoners	1.000000	0.995671	0.996897
Aantal_0_15	0.995671	1.000000	0.989825
Aantal_15_25	0.996897	0.989825	1.000000

Toelichting

De correlatiematrix laat zien dat er een zeer sterke positieve relatie bestaat tussen het totaal aantal inwoners en het aantal jongeren in beide leeftijdsgroepen. De correlaties liggen allemaal boven de 0.98. Dit betekent dat wijken met meer inwoners vrijwel altijd ook meer kinderen en jongeren hebben.

Dit bevestigt dat de omvang van de bevolking een belangrijke factor is bij het verklaren van het aantal jongeren in een wijk. Tegelijkertijd betekent deze sterke samenhang ook dat sommige variabelen mogelijk vergelijkbare informatie bevatten, wat belangrijk is om rekening mee te houden bij het bouwen van een model.

2.2.7 Percentage jeugd berekenen (Ismael + Wail)

In deze stap berekenen we het percentage kinderen (0–15 jaar) en jongeren (15–25 jaar) ten opzichte van het totaal aantal inwoners per wijk. Daarnaast berekenen we het totale percentage jeugd (0–25 jaar). Deze percentages maken het mogelijk om wijken beter met elkaar te vergelijken, ongeacht verschillen in totale bevolkingsgrootte.

[15]:
```
# dataset opnieuw laden (zeker dat df bestaat)
import pandas as pd

df = pd.read_csv("../../data/processed/df_v8_ready_for_analysis.csv")

# percentages berekenen
df["pct_0_15"] = (df["Aantal_0_15"] / df["AantalInwoners"]) * 100
df["pct_15_25"] = (df["Aantal_15_25"] / df["AantalInwoners"]) * 100
df["Perc_TotaleJeugd"] = df["pct_0_15"] + df["pct_15_25"]

df[["pct_0_15", "pct_15_25", "Perc_TotaleJeugd"]].describe()

# Onmogelijke waarden (>100%)
df[df["Perc_TotaleJeugd"] > 100][
    ["Wijk", "WijkCode", "AantalInwoners", "Aantal_0_15", "Aantal_15_25",␣
 ↪"Perc_TotaleJeugd"]
].head(10)

# Extra check: jeugd groter dan totaal (moet 0 zijn)
df[(df["Aantal_0_15"] + df["Aantal_15_25"]) > df["AantalInwoners"]].head(10)

# Filter onmogelijke rijen weg
df = df[df["Perc_TotaleJeugd"] <= 100].copy()
print("Shape na filteren onmogelijke waarden:", df.shape)
df.head()
```

```
Shape na filteren onmogelijke waarden: (236, 10)
```

[15]:
```
     Peildatum  Jaar  WijkCode        Wijk  AantalInwoners  Aantal_0_15  \
0  2018-01-01  2018  WK003401  WK003401             22790         3975
1  2019-01-01  2019  WK003401  WK003401             22875         3945
2  2020-01-01  2020  WK003401  WK003401             23275         4005
```

```
3  2021-01-01  2021  WK003401  WK003401             23530         4180
4  2022-01-01  2022  WK003401  WK003401              1635          175

   Aantal_15_25   pct_0_15  pct_15_25  Perc_TotaleJeugd
0          2420  17.441860  10.618692         28.060553
1          2415  17.245902  10.557377         27.803279
2          2485  17.207304  10.676692         27.883996
3          2465  17.764556  10.475988         28.240544
4           150  10.703364   9.174312         19.877676
```

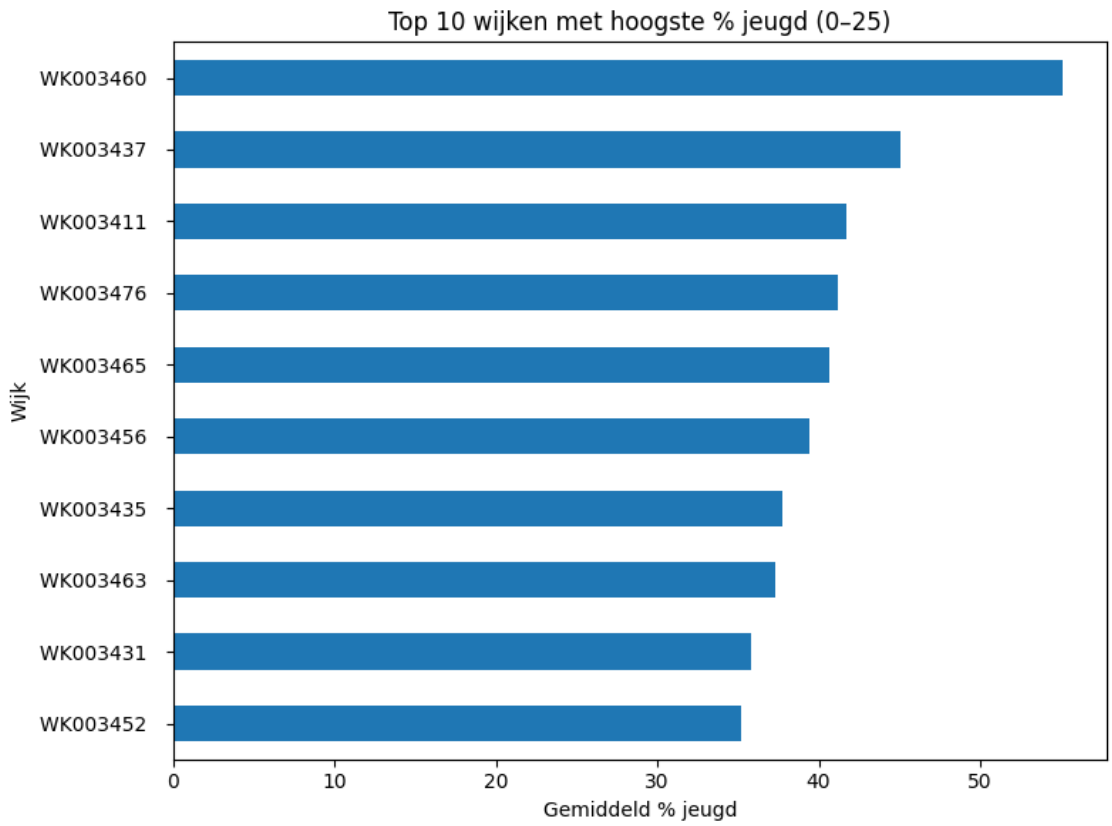
Toelichting

Na het berekenen van de percentages worden controles uitgevoerd om te kijken of er onmogelijke waarden voorkomen, zoals een totaal jeugdpercentage boven de 100% of situaties waarin het aantal jongeren groter is dan het totaal aantal inwoners. Er wordt één ongeldige rij gevonden en verwijderd. Na deze opschoning bevat de dataset 236 rijen en is deze geschikt voor verdere analyse.

2.2.8 Top 10 wijken met hoogste jeugdpercentage (Wail)

In deze stap berekenen we per wijk het gemiddelde percentage jeugd (0–25 jaar). Vervolgens selecteren we de tien wijken met het hoogste aandeel jongeren. Dit helpt om te zien in welke wijken relatief veel jongeren wonen.

[16]:
```
gemiddeld_wijk = df.groupby("Wijk")["Perc_TotaleJeugd"].mean()
top10_wijk = gemiddeld_wijk.sort_values(ascending=False).head(10)

plt.figure(figsize=(8,6))
top10_wijk.sort_values().plot(kind="barh")
plt.title("Top 10 wijken met hoogste % jeugd (0-25)")
plt.xlabel("Gemiddeld % jeugd")
plt.tight_layout()
plt.show()
```



Toelichting

De grafiek laat zien dat sommige wijken een relatief hoog aandeel jongeren hebben in vergelijking met andere wijken. In deze top 10 bestaat meer dan ongeveer 35% tot ruim 50% van de bevolking uit jongeren van 0–25 jaar. Dit betekent dat deze wijken relatief jong zijn en mogelijk een grotere behoefte hebben aan voorzieningen zoals scholen, sportfaciliteiten en jeugdzorg.

Tijdreeks per wijk (Bo + Ismael)

In deze stap berekenen we het totale aantal jongeren (0–25 jaar) per wijk door het aantal kinderen (0–15 jaar) en jongeren (15–25 jaar) bij elkaar op te tellen. Vervolgens bepalen we welke tien wijken gemiddeld de meeste jongeren hebben.

[17]:
```
# Totale jeugd berekenen
df["Totale_Jeugd"] = df["Aantal_0_15"] + df["Aantal_15_25"]

# Top 10 wijken bepalen
top10_wijken = (
    df.groupby("Wijk")["Totale_Jeugd"]
    .mean()
    .sort_values(ascending=False)
    .head(10)
```

```
    .index
)

print("Top 10 wijken:", list(top10_wijken))
```

```
Top 10 wijken: ['WK003402  ', 'WK003403  ', 'WK003401  ', 'WK003427  ',
'WK003424  ', 'WK003430  ', 'WK003418  ', 'WK003428  ', 'WK003404  ', 'WK003463
']
```

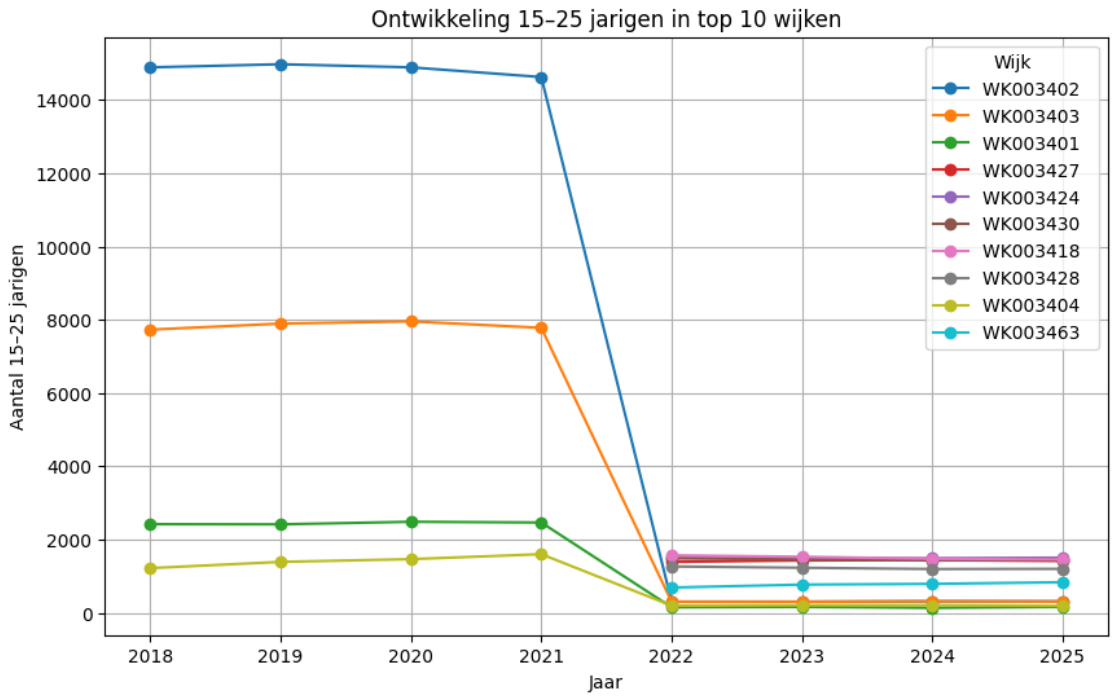
Toelichting

De resultaten laten zien welke wijken gemiddeld de grootste jeugdpopulatie hebben. Dit zijn voornamelijk de grotere wijken in Almere waar relatief veel inwoners wonen. Omdat deze wijken ook meer jongeren hebben, kunnen ze een grotere vraag hebben naar voorzieningen zoals scholen, sportfaciliteiten en jeugdzorg.

2.2.9 Ontwikkeling van 15–25 jarigen in de top 10 wijken

In deze stap analyseren we hoe het aantal jongeren van 15–25 jaar zich ontwikkelt door de jaren heen in de tien wijken met de grootste jeugdpopulatie. Door deze ontwikkeling per wijk te visualiseren kunnen we trends en veranderingen in de jeugdpopulatie beter begrijpen.

[18]:
```
plt.figure(figsize=(10,6))

for wijk in top10_wijken:

    subset = df[df["Wijk"] == wijk].sort_values("Jaar")

    plt.plot(
        subset["Jaar"],
        subset["Aantal_15_25"],
        marker="o",
        label=wijk
    )

plt.xlabel("Jaar")
plt.ylabel("Aantal 15-25 jarigen")
plt.title("Ontwikkeling 15-25 jarigen in top 10 wijken")

plt.legend(title="Wijk")
plt.grid(True)

plt.show()
```



Toelichting

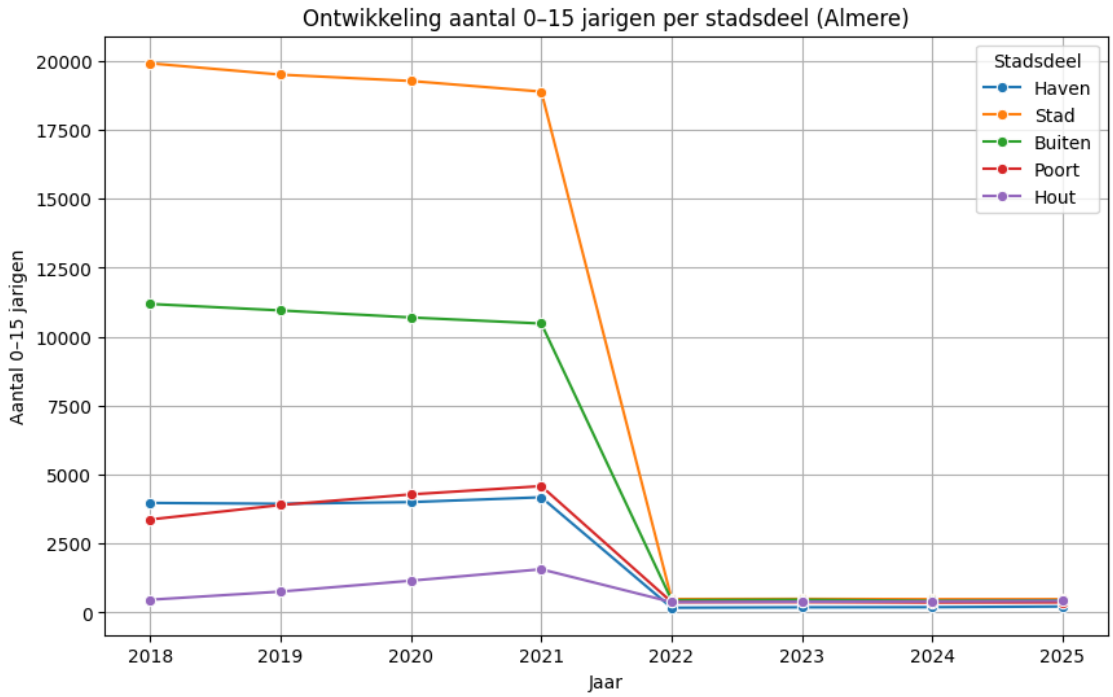
De grafiek laat zien dat sommige wijken een duidelijk hoger aantal jongeren hebben dan andere wijken. Daarnaast is te zien dat er vanaf 2022 een sterke daling zichtbaar is in enkele wijken. Dit kan komen door veranderingen in de wijkindeling of doordat nieuwe, kleinere wijken in de dataset zijn opgenomen. Hierdoor lijken de aantallen per wijk lager, terwijl de totale jeugdpopulatie in Almere niet per se is afgenomen. De grafiek laat vooral zien dat de ontwikkeling van jongeren per wijk sterk kan verschillen.

2.2.10 Bo's stadsdeel mapping (alleen als WijkCode WK003401 etc.)

In deze stap worden de wijken gekoppeld aan de stadsdelen van Almere. Vervolgens analyseren we hoe het aantal kinderen van 0–15 jaar zich ontwikkelt per stadsdeel over de jaren. Dit helpt om verschillen in jeugdontwikkeling tussen de verschillende delen van de stad zichtbaar te maken.

[19]:
```
# Mapping wijkcodes -> stadsdelen
wijk_mapping = {
    "WK003401": "Haven",
    "WK003402": "Stad",
    "WK003403": "Buiten",
    "WK003404": "Poort",
    "WK003405": "Hout",
}

df["Stadsdeel"] = df["WijkCode"].map(wijk_mapping)
```

```
df_map = df.dropna(subset=["Stadsdeel"]).copy()

plt.figure(figsize=(10,6))

sns.lineplot(
    data=df_map,
    x="Jaar",
    y="Aantal_0_15",
    hue="Stadsdeel",
    marker="o"
)

plt.title("Ontwikkeling aantal 0-15 jarigen per stadsdeel (Almere)")
plt.xlabel("Jaar")
plt.ylabel("Aantal 0-15 jarigen")

plt.grid(True)

plt.legend(title="Stadsdeel")

plt.show()
```



Toelichting

De grafiek laat zien dat de stadsdelen duidelijke verschillen hebben in het aantal kinderen van 0–15 jaar. Stadsdeel Stad en Buiten hebben het hoogste aantal kinderen, terwijl Hout duidelijk minder kinderen heeft. Daarnaast is vanaf 2022 een sterke daling zichtbaar. Dit komt waarschijnlijk doordat de wijkindeling in de dataset verandert, waardoor de aantallen per stadsdeel anders worden verdeeld. Ondanks deze verandering blijft zichtbaar dat sommige stadsdelen structureel meer jongeren hebben dan andere.

---

3 Fase 3 – Data Preparation

In deze fase wordt de dataset voorbereid voor modellering. De analyseklare dataset uit fase 2 wordt uitgebreid met nieuwe features die gebruikt kunnen worden in het regressiemodel.

De input van deze fase is:

`../../data/processed/df_v8_ready_for_analysis.csv`

Het doel is om de dataset model-klaar te maken. Binnen deze fase zijn door verschillende teamleden aanvullende features en controles uitgevoerd. Deze onderdelen worden hieronder stap voor stap samengebracht.

De uiteindelijke output van deze fase is:

`../../data/processed/df_v12_model_ready.csv`

3.1 3.1 Basisdataset laden

We starten vanuit de analyseklare dataset uit fase 2. Deze dataset bevat per wijk en per jaar de belangrijkste demografische variabelen die als basis dienen voor verdere feature engineering.

[20]:
```
import pandas as pd
from pathlib import Path

base_path = Path("../../data/processed/df_v8_ready_for_analysis.csv")
df_base = pd.read_csv(base_path)

df_base.head()
```

[20]:
```
    Peildatum  Jaar  WijkCode        Wijk  AantalInwoners  Aantal_0_15  \
0  2018-01-01  2018  WK003401  WK003401             22790         3975
1  2019-01-01  2019  WK003401  WK003401             22875         3945
2  2020-01-01  2020  WK003401  WK003401             23275         4005
3  2021-01-01  2021  WK003401  WK003401             23530         4180
4  2022-01-01  2022  WK003401  WK003401              1635          175

   Aantal_15_25   pct_0_15  pct_15_25
0          2420  17.441860  10.618692
1          2415  17.245902  10.557377
2          2485  17.207304  10.676692
3          2465  17.764556  10.475988
```

```
4            150  10.703364   9.174312
```

Toelichting

In deze stap laden we de analyseklare dataset die eerder in het project is opgeslagen. Deze dataset bevat per wijk en per jaar de belangrijkste variabelen, zoals het totaal aantal inwoners en het aantal inwoners in de leeftijdsgroepen 0–15 jaar en 15–25 jaar. Deze gegevens vormen de basis voor de verdere stappen in de data preparation en modellering.

3.1.1 Controle van sleutelvariabelen

In deze stap bekijken we de variabelen Wijk en Jaar. Deze twee kolommen vormen samen de unieke sleutel van de dataset, omdat elke rij een specifieke wijk in een bepaald jaar vertegenwoordigt.

[21]:
```
df_base[["Wijk", "Jaar"]].head()
```

[21]:
```
         Wijk  Jaar
0  WK003401    2018
1  WK003401    2019
2  WK003401    2020
3  WK003401    2021
4  WK003401    2022
```

3.1.2 Duplicaten check

Hier kijken we of er duplicaten zijn.

[22]:
```
df_base.duplicated(subset=["Wijk", "Jaar"]).sum()
```

[22]:
```
0
```

Toelichting

De controle op `Wijk` + `Jaar` laat zien of iedere combinatie uniek voorkomt. Dit is belangrijk, omdat de verschillende feature-datasets later op deze sleutel worden samengevoegd.

3.2 3.2 Ratio jongeren ten opzichte van totale bevolking (Ismael)

In deze stap worden relatieve indicatoren berekend om wijken eerlijker met elkaar te kunnen vergelijken. Absolute aantallen jongeren kunnen stijgen doordat de totale bevolking groeit, maar zeggen minder over de werkelijke demografische samenstelling. Daarom wordt het aantal jongeren gedeeld door het totaal aantal inwoners.

[23]:
```
df = pd.read_csv("../../data/processed/df_v8_ready_for_analysis.csv")
df.columns = df.columns.astype(str).str.strip()

# Peildatum -> Jaar
df["Peildatum"] = pd.to_datetime(df["Peildatum"], dayfirst=True,␣
 ↪errors="coerce")
df["Jaar"] = df["Peildatum"].dt.year
```

```
# Alleen consistente jaren
df = df[df["Jaar"] <= 2021].copy()

# wijkcodes opschonen
df["Wijk"] = df["Wijk"].astype(str).str.strip()

# Ratio's
df["Ratio_0_15"] = df["Aantal_0_15"] / df["AantalInwoners"]
df["Ratio_15_25"] = df["Aantal_15_25"] / df["AantalInwoners"]

# Checks
print("Min/Max Ratio 0-15:", df["Ratio_0_15"].min(), df["Ratio_0_15"].max())
print("Min/Max Ratio 15-25:", df["Ratio_15_25"].min(), df["Ratio_15_25"].max())
```

```
Min/Max Ratio 0-15: 0.17207303974221266 0.25566825775656327
Min/Max Ratio 15-25: 0.08333333333333333 0.13963489555906616
```

Toelichting

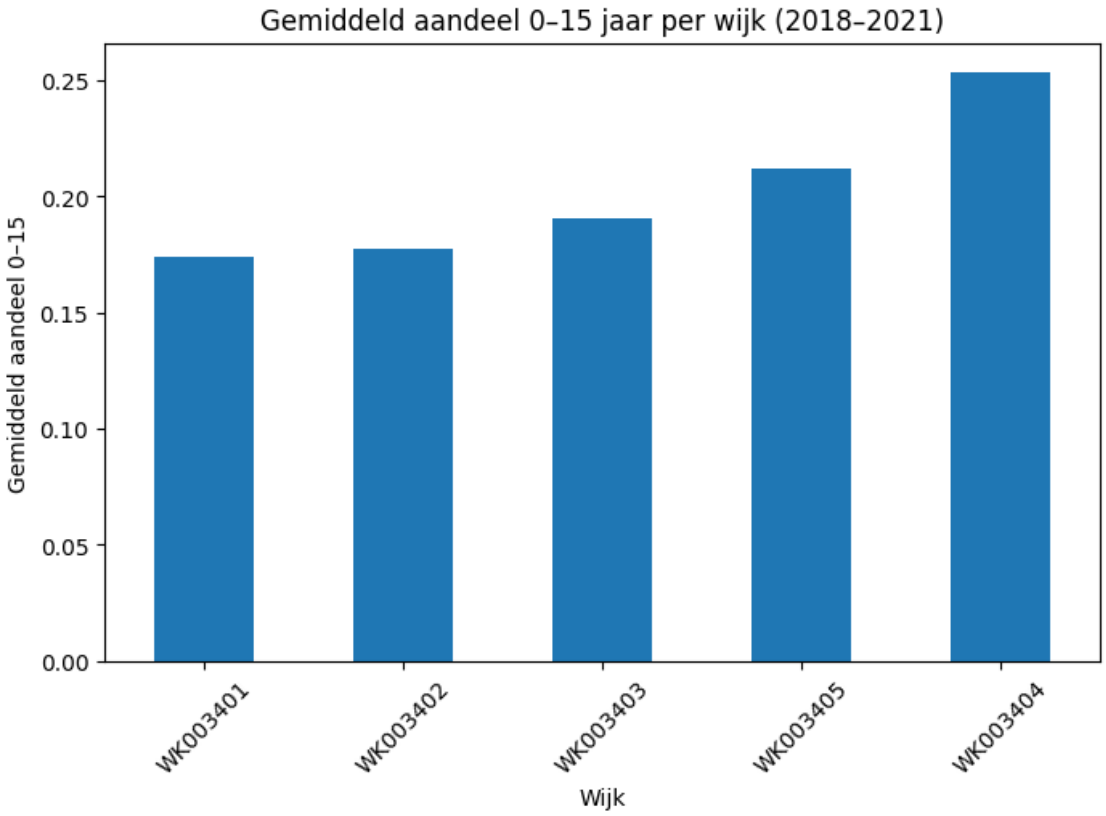
In deze stap worden ratio's berekend om het aandeel jongeren binnen de totale bevolking van een wijk te bepalen. De ratio's geven aan welk deel van de inwoners bestaat uit kinderen van 0–15 jaar en jongeren van 15–25 jaar.

De resultaten laten zien dat het aandeel kinderen van 0–15 jaar in de wijken ongeveer tussen 17% en 26% ligt. Voor de leeftijdsgroep 15–25 jaar ligt dit aandeel ongeveer tussen 8% en 14%. Deze waarden lijken realistisch en bevestigen dat de berekende ratio's geschikt zijn om verschillen tussen wijken beter te analyseren.

3.2.1 Gemiddeld aandeel kinderen (0–15 jaar) per wijk

In deze stap analyseren we het gemiddelde aandeel kinderen van 0–15 jaar per wijk. Door de ratio van deze leeftijdsgroep te berekenen en per wijk te middelen, kunnen we zien in welke wijken relatief meer kinderen wonen.

[24]:
```
gemiddeld_0 = df.groupby("Wijk")["Ratio_0_15"].mean().sort_values()

plt.figure(figsize=(8,5))
gemiddeld_0.plot(kind="bar")

plt.ylabel("Gemiddeld aandeel 0-15")
plt.title("Gemiddeld aandeel 0-15 jaar per wijk (2018-2021)")
plt.xticks(rotation=45)
plt.show()
```



Toelichting

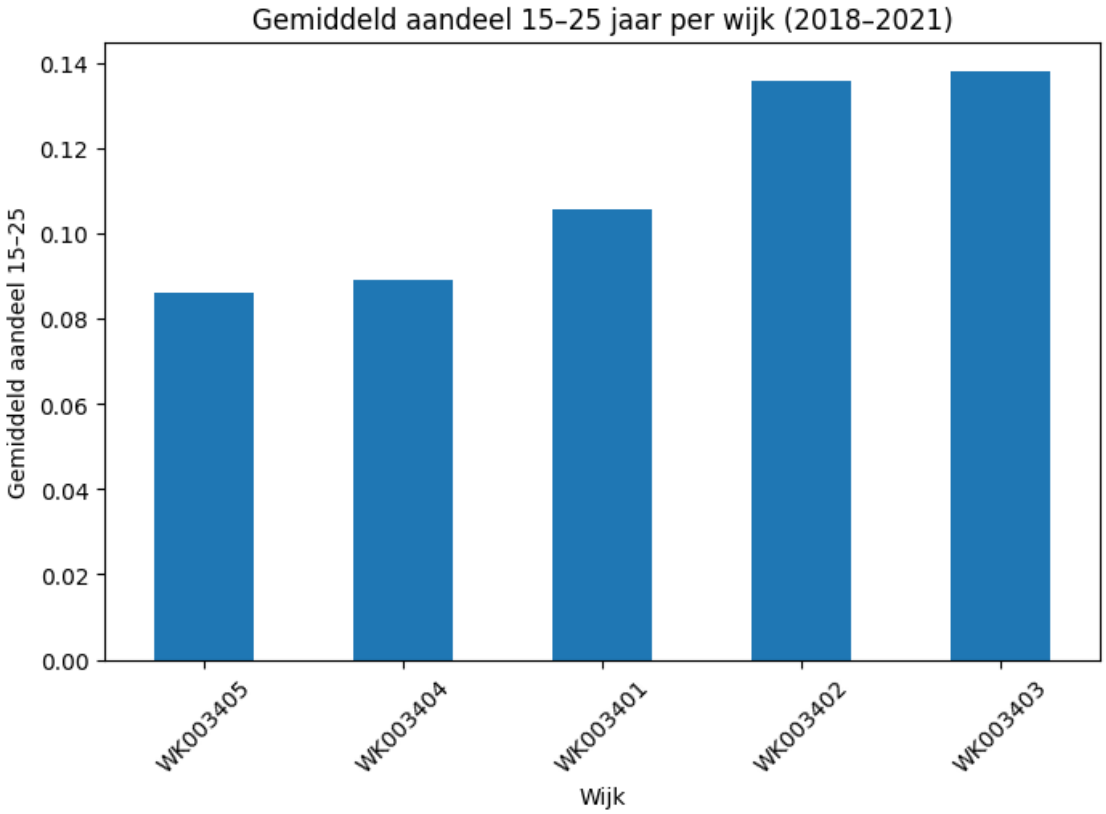
De grafiek laat zien dat het aandeel kinderen van 0–15 jaar per wijk verschilt. Sommige wijken hebben een duidelijk hoger aandeel kinderen dan andere wijken. Dit betekent dat bepaalde wijken een relatief jongere bevolking hebben. Deze verschillen kunnen belangrijk zijn voor beleidsplanning, omdat wijken met meer kinderen mogelijk meer behoefte hebben aan voorzieningen zoals scholen en sportfaciliteiten.

3.2.2 Gemiddeld aandeel kinderen (0–15 jaar) per wijk

In deze stap analyseren we het gemiddelde aandeel kinderen van 0–15 jaar per wijk. Door de ratio van deze leeftijdsgroep te berekenen en per wijk te middelen, kunnen we zien in welke wijken relatief meer kinderen wonen.

[25]:
```
gemiddeld_1 = df.groupby("Wijk")["Ratio_15_25"].mean().sort_values()

plt.figure(figsize=(8,5))
gemiddeld_1.plot(kind="bar")

plt.ylabel("Gemiddeld aandeel 15-25")
plt.title("Gemiddeld aandeel 15-25 jaar per wijk (2018-2021)")
plt.xticks(rotation=45)
```

```
plt.show()
```



Toelichting

De grafiek laat zien dat het aandeel kinderen van 0–15 jaar per wijk verschilt. Sommige wijken hebben een duidelijk hoger aandeel kinderen dan andere wijken. Dit betekent dat bepaalde wijken een relatief jongere bevolking hebben. Deze verschillen kunnen belangrijk zijn voor beleidsplanning, omdat wijken met meer kinderen mogelijk meer behoefte hebben aan voorzieningen zoals scholen en sportfaciliteiten.

3.2.3 Ratio-features

Aandeel jongeren in verhouding tot het gehele bevolking.

[26]:
```
df.to_csv("../../data/processed/df_v10_ratio_temp.csv", index=False)
```

De ratio-features laten het aandeel jongeren binnen de totale bevolking zien. Daarmee kunnen verschillen tussen wijken beter worden geïnterpreteerd dan met alleen absolute aantallen.

3.3 3.3 Stabiliteit en plausibiliteit van ratio-features controleren (Ismael)

Na het berekenen van de ratio's wordt gecontroleerd of deze nieuwe variabelen technisch correct en inhoudelijk plausibel zijn. Hierbij wordt gekeken naar onmogelijke waarden, null-waarden en

deling door nul.

[27]:
```
df = pd.read_csv("../../data/processed/df_v10_ratio_temp.csv")
df.columns = df.columns.astype(str).str.strip()

print("Start aantal rijen:", len(df))

print("\nControle ratio > 1:")
print("Ratio_0_15 > 1:", (df["Ratio_0_15"] > 1).sum())
print("Ratio_15_25 > 1:", (df["Ratio_15_25"] > 1).sum())

print("\nControle ratio < 0:")
print("Ratio_0_15 < 0:", (df["Ratio_0_15"] < 0).sum())
print("Ratio_15_25 < 0:", (df["Ratio_15_25"] < 0).sum())

print("\nAantal NaN waarden:")
print(df[["Ratio_0_15", "Ratio_15_25"]].isna().sum())

print("\nAantal rijen met inwoners = 0:")
print((df["AantalInwoners"] == 0).sum())
```

```
Start aantal rijen: 20

Controle ratio > 1:
Ratio_0_15 > 1: 0
Ratio_15_25 > 1: 0

Controle ratio < 0:
Ratio_0_15 < 0: 0
Ratio_15_25 < 0: 0

Aantal NaN waarden:
Ratio_0_15     0
Ratio_15_25    0
dtype: int64

Aantal rijen met inwoners = 0:
0
```

Toelichting

De controles laten zien dat de dataset 20 rijen bevat en dat alle ratio's binnen een logische range liggen. Er komen geen waarden groter dan 1 of kleiner dan 0 voor. Daarnaast zijn er geen missende waarden en zijn er geen rijen waarbij het aantal inwoners gelijk is aan nul. Dit betekent dat de berekende ratio-variabelen consistent en geschikt zijn voor verdere analyse.

3.3.1 Opschonen van de ratio-dataset

In deze stap wordt de dataset gefilterd op logische waarden. Alleen rijen waarbij het aantal inwoners groter is dan nul en waarbij de ratio's van de leeftijdsgroepen tussen 0 en 1 liggen worden behouden.

[28]:
```
df_clean = df[
    (df["AantalInwoners"] > 0) &
    (df["Ratio_0_15"].between(0,1)) &
    (df["Ratio_15_25"].between(0,1))
].copy()

print("\nAantal rijen na cleaning:", len(df_clean))
```

```
Aantal rijen na cleaning: 20
```

Opschonen van de ratio-dataset

In deze stap wordt de dataset gefilterd op logische waarden. Alleen rijen waarbij het aantal inwoners groter is dan nul en waarbij de ratio's van de leeftijdsgroepen tussen 0 en 1 liggen worden behouden.

Uit deze controle blijkt of de berekende ratio-features bruikbaar zijn voor verdere analyse en modellering.

3.4 3.4 Lag-feature maken op basis van vorige waarde (Wail)

In deze stap wordt een lag-feature gemaakt. Een lag-feature gebruikt een waarde van het vorige jaar als voorspeller voor het huidige jaar. Dit kan helpen om trends in de data beter te modelleren.

In deze stap maken we een nieuwe variabele die het totale aantal jongeren (0–25 jaar) per wijk berekent. Daarna wordt een lag-feature toegevoegd. Deze variabele bevat de waarde van het aantal jongeren uit het vorige jaar en kan later worden gebruikt als voorspeller in het model.

[29]:
```
import pandas as pd

input_path = "../../data/processed/df_v8_ready_for_analysis.csv"
df = pd.read_csv(input_path)

print(df.shape)
df.head()
```

```
(237, 9)
```

[29]:
```
     Peildatum  Jaar  WijkCode        Wijk  AantalInwoners  Aantal_0_15  \
0  2018-01-01  2018  WK003401  WK003401             22790         3975
1  2019-01-01  2019  WK003401  WK003401             22875         3945
2  2020-01-01  2020  WK003401  WK003401             23275         4005
3  2021-01-01  2021  WK003401  WK003401             23530         4180
4  2022-01-01  2022  WK003401  WK003401              1635          175

   Aantal_15_25   pct_0_15  pct_15_25
```

```
0          2420  17.441860  10.618692
1          2415  17.245902  10.557377
2          2485  17.207304  10.676692
3          2465  17.764556  10.475988
4           150  10.703364   9.174312
```

[30]:
```python
df = df.sort_values(["Wijk", "Jaar"])

df["Jeugdzorg"] = df["Aantal_0_15"] + df["Aantal_15_25"]

df["Jeugdzorg_lag1"] = df.groupby("Wijk")["Jeugdzorg"].shift(1)

df.head()
```

[30]:
```
    Peildatum  Jaar  WijkCode        Wijk  AantalInwoners  Aantal_0_15  \
0  2018-01-01  2018  WK003401  WK003401             22790         3975
1  2019-01-01  2019  WK003401  WK003401             22875         3945
2  2020-01-01  2020  WK003401  WK003401             23275         4005
3  2021-01-01  2021  WK003401  WK003401             23530         4180
4  2022-01-01  2022  WK003401  WK003401              1635          175

   Aantal_15_25   pct_0_15  pct_15_25  Jeugdzorg  Jeugdzorg_lag1
0          2420  17.441860  10.618692       6395             NaN
1          2415  17.245902  10.557377       6360          6395.0
2          2485  17.207304  10.676692       6490          6360.0
3          2465  17.764556  10.475988       6645          6490.0
4           150  10.703364   9.174312        325          6645.0
```

[31]:
```python
df = df.dropna(subset=["Jeugdzorg_lag1"])

print(df.shape)
```

```
(181, 11)
```

[32]:
```python
output_path = "../../data/processed/df_v12_model_ready.csv"
df.to_csv(output_path, index=False)

print("Dataset opgeslagen:", output_path)
```

```
Dataset opgeslagen: ../../data/processed/df_v12_model_ready.csv
```

Met `groupby("Wijk")` wordt de lag alleen binnen dezelfde wijk berekend. De functie `shift(1)` verschuift de waarden één jaar naar beneden, zodat de waarde van het vorige jaar beschikbaar wordt als nieuwe variabele.

Toelichting

De nieuwe variabele Jeugdzorg is berekend door het aantal inwoners van 0–15 jaar en 15–25 jaar op te tellen. Vervolgens is met behulp van groupby en shift(1) de waarde van het vorige jaar per wijk toegevoegd als Jeugdzorg_lag1. Voor het eerste jaar per wijk ontbreekt deze waarde, waardoor deze

rijen worden verwijderd. De uiteindelijke dataset wordt opgeslagen als df_v12_model_ready.csv en is klaar voor gebruik in het voorspellende model.

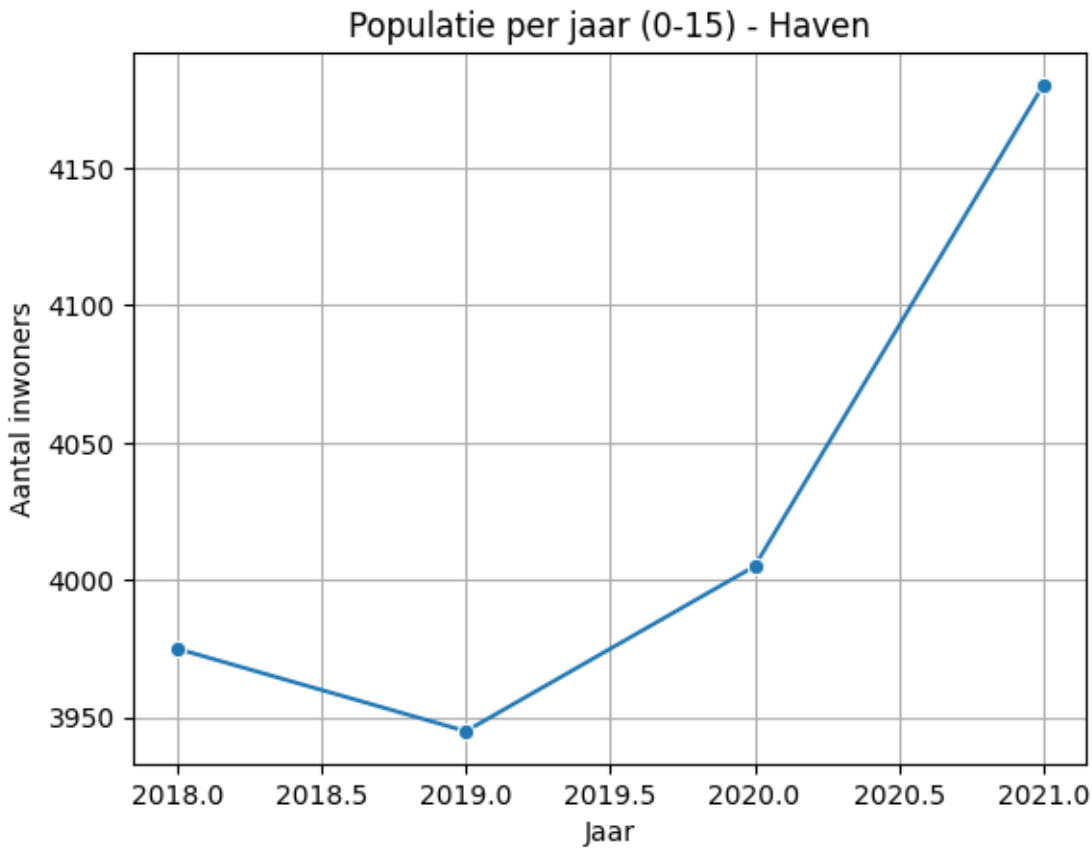
3.5 3.5 Groei van leeftijdsgroepen per wijk analyseren (Bo)

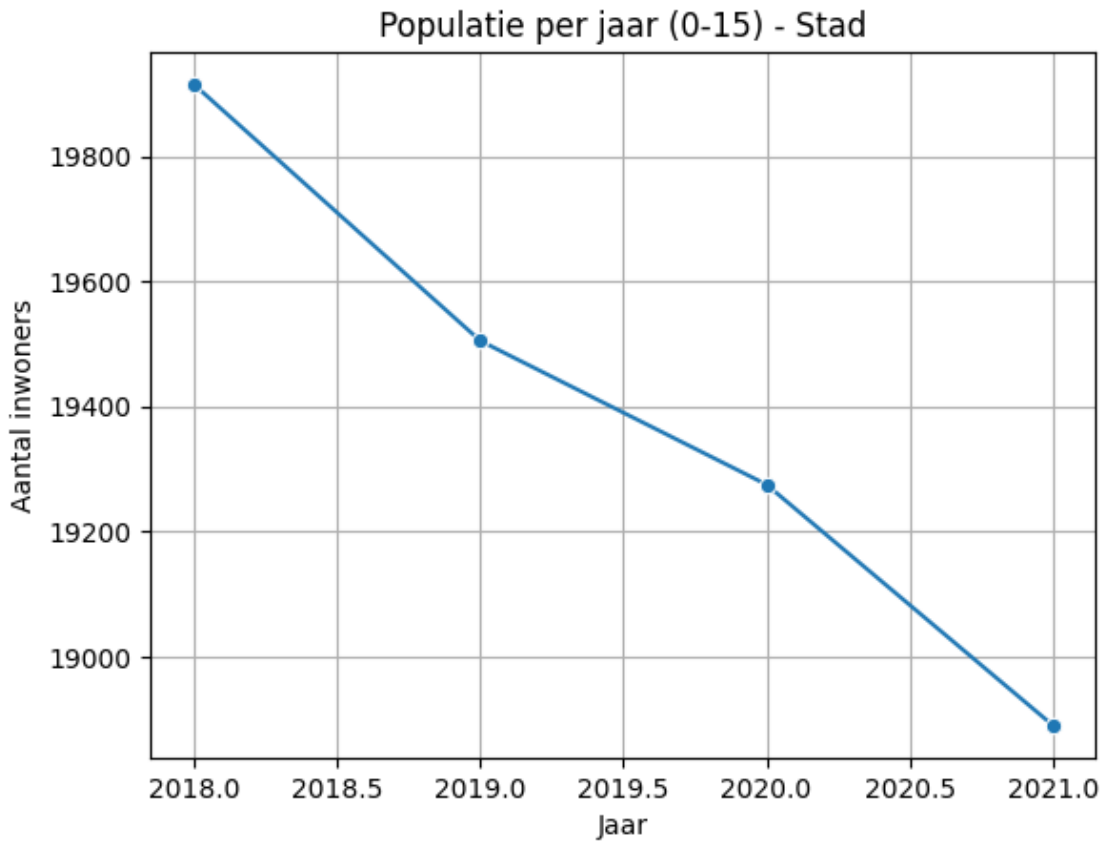
In dit onderdeel wordt gekeken naar de ontwikkeling van de leeftijdsgroepen 0–15 jaar en 15–25 jaar tussen 2018 en 2021. De groei wordt zowel absoluut als procentueel weergegeven.

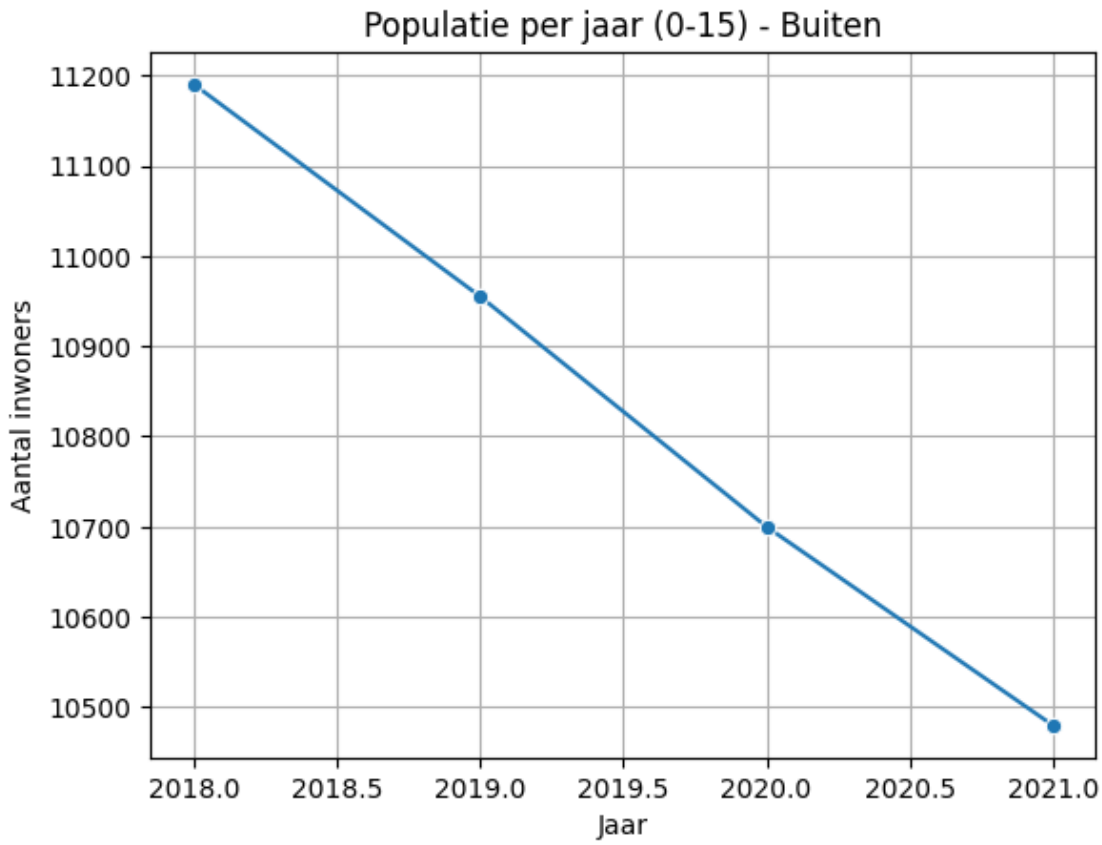
In deze stap analyseren we hoe de leeftijdsgroepen 0–15 jaar en 15–25 jaar zich ontwikkelen tussen 2018 en 2021 in de verschillende stadsdelen van Almere. Eerst worden de wijken gekoppeld aan hun stadsdeel. Daarna worden trends per jaar gevisualiseerd en wordt zowel de absolute groei als de procentuele groei van de leeftijdsgroepen berekend.

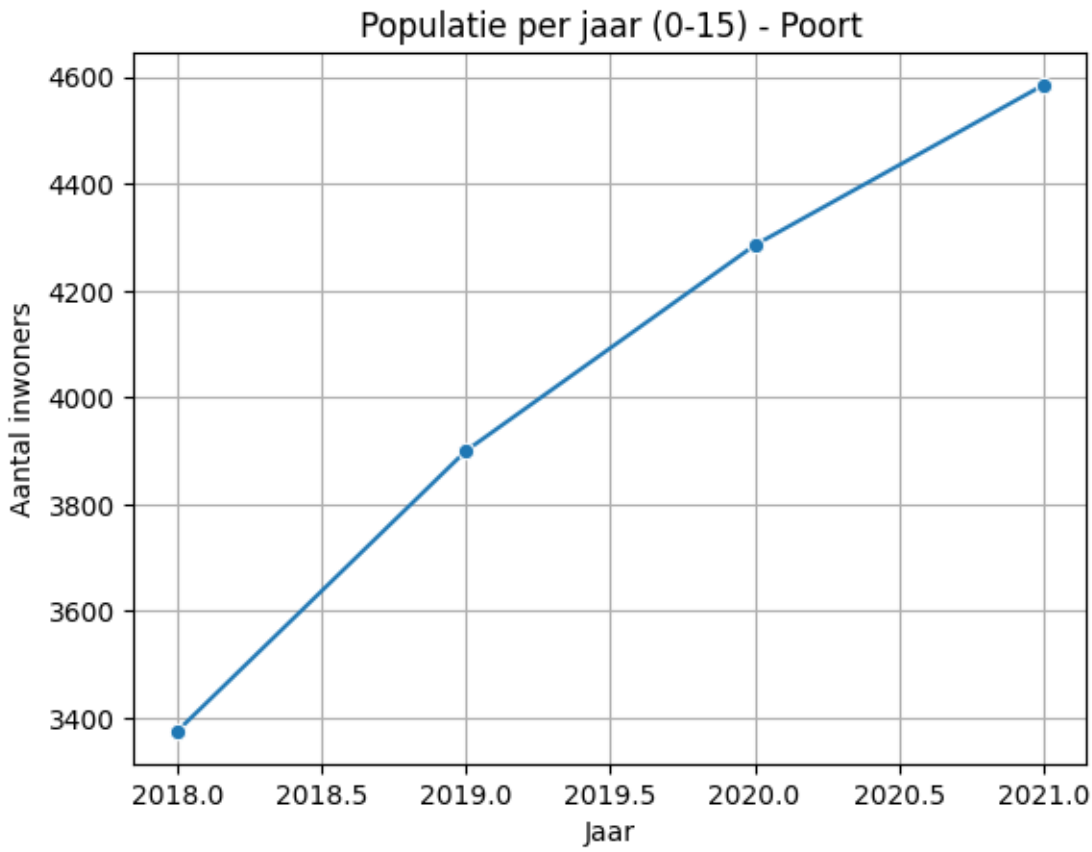
[33]:
```
df_groei = pd.read_csv('../../data/processed/df_v10_ratio_temp.csv')
wijk_mapping = {
    "WK003401": "Haven",
    "WK003402": "Stad",
    "WK003403": "Buiten",
    "WK003404": "Poort",
    "WK003405": "Hout",
}

df_groei["Stadsdeel"] = df_groei["Wijk"].map(wijk_mapping)
```

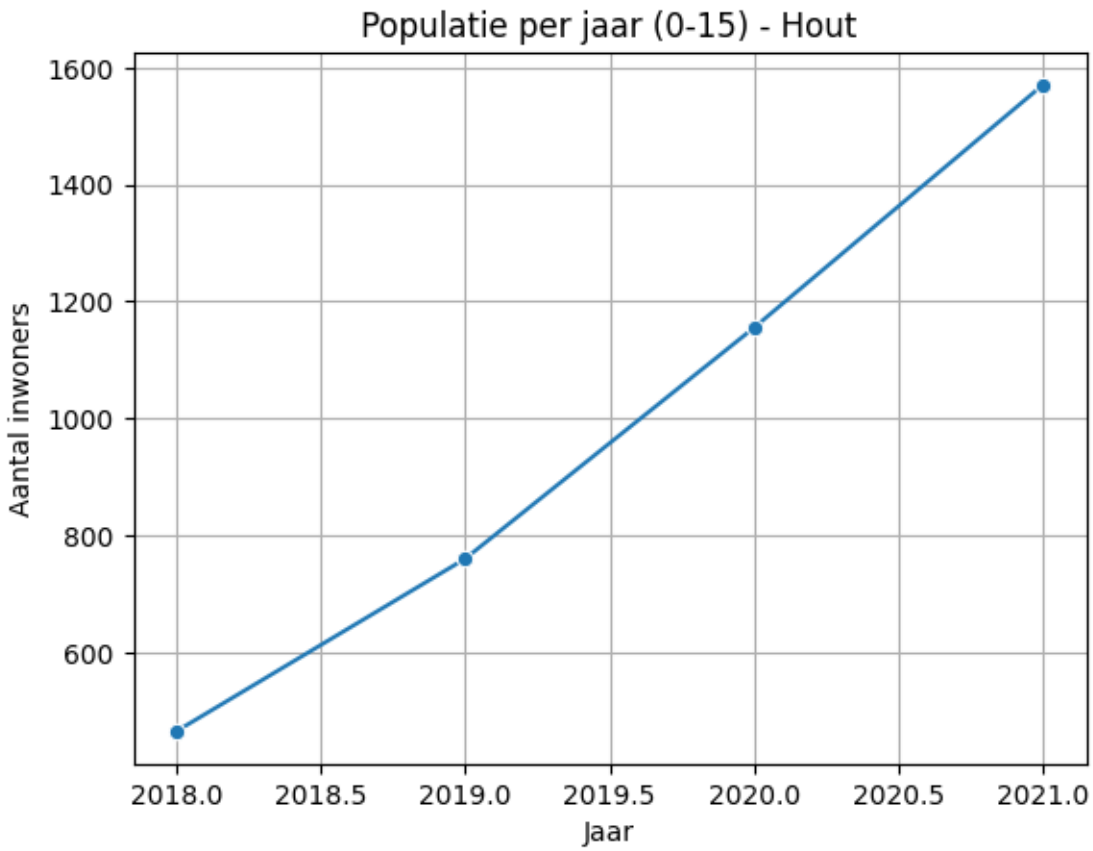
[34]:
```
import matplotlib.pyplot as plt
import seaborn as sns

for wijk in df_groei["Stadsdeel"].unique():
    plt.figure()

    subset1 = df_groei[df_groei["Stadsdeel"] == wijk]

    ax = sns.lineplot(
        data=subset1,
        x="Jaar",
        y="Aantal_0_15",
        marker="o"
    )

    ax.grid(True)
    plt.title(f"Populatie per jaar (0-15) - {wijk}")
    plt.xlabel("Jaar")
    plt.ylabel("Aantal inwoners")
    plt.show()
```


Populatie per jaar (0-15) - Haven
Aantal inwoners
4150
4100
4050
4000
3950
2018.0
2018.5
2019.0
2019.5
2020.0
2020.5
2021.0
Jaar


Populatie per jaar (0-15) - Stad
Aantal inwoners
19800
19600
19400
19200
19000
2018.0
2018.5
2019.0
2019.5
2020.0
2020.5
2021.0
Jaar

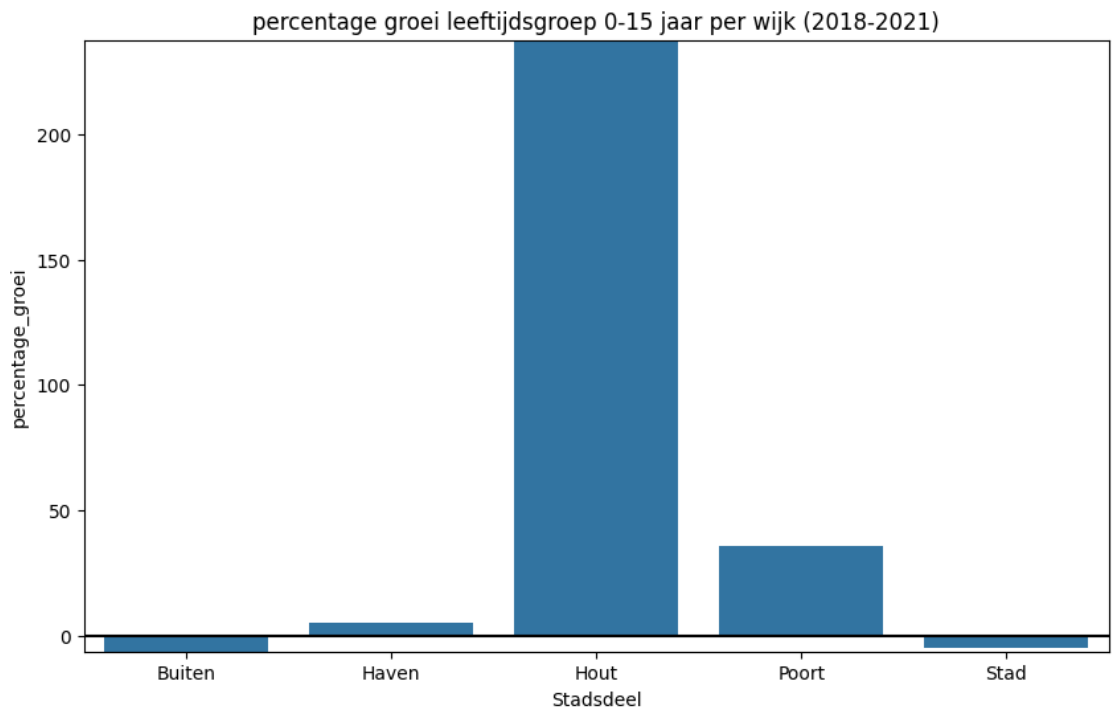

Populatie per jaar (0-15) - Buiten
Aantal inwoners
11200
11100
11000
10900
10800
10700
10600
10500
2018.0
2018.5
2019.0
2019.5
2020.0
2020.5
2021.0
Jaar


Populatie per jaar (0-15) - Poort
Aantal inwoners
4600
4400
4200
4000
3800
3600
3400
2018.0
2018.5
2019.0
2019.5
2020.0
2020.5
2021.0
Jaar

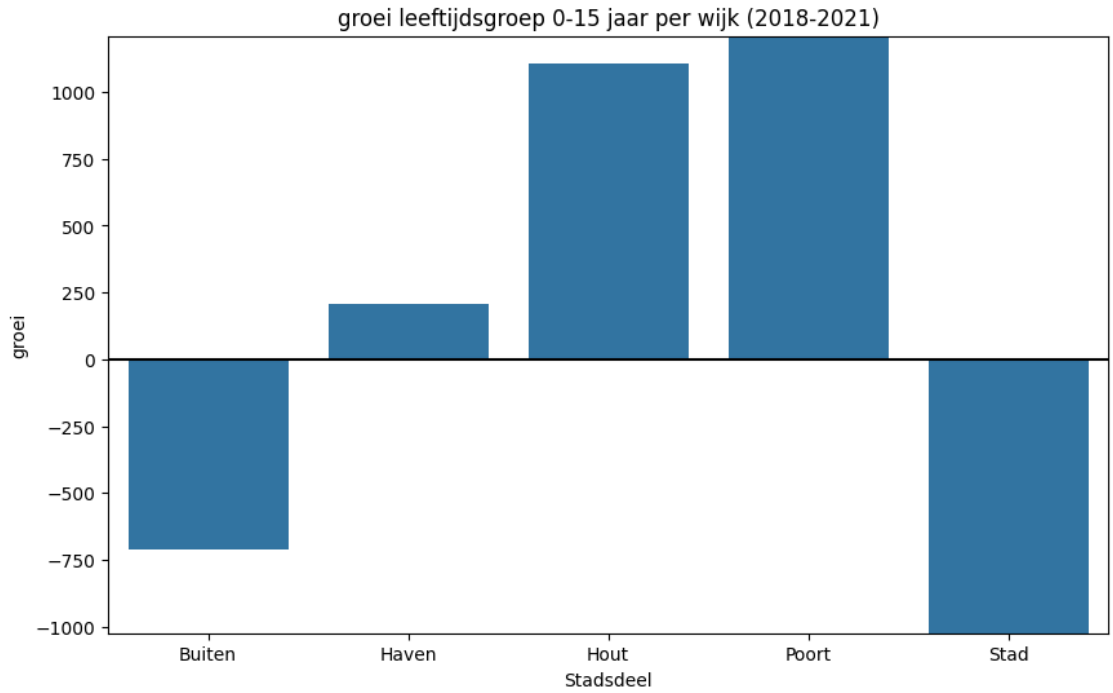


[35]:
```
subset2 = df_groei

subset2['Peildatum'] = pd.to_datetime(subset2['Peildatum'])

subset2 = subset2[subset2['Jaar'].isin([2018, 2021])]

subset2 = subset2.pivot(index='Stadsdeel', columns='Jaar', values='Aantal_0_15')

subset2["groei"] = subset2[2021] - subset2[2018]

subset2["percentage_groei"] = (subset2["groei"] / subset2[2018]) * 100
plt.figure(figsize=(10, 6))
ax = sns.barplot(
    data=subset2.reset_index(),
    x='Stadsdeel',
    y='percentage_groei',
)

plt.title('percentage groei leeftijdsgroep 0-15 jaar per wijk (2018-2021)')
```

```
ax.axhline(0, color='black')
ax.set_ylim(subset2['percentage_groei'].min(), subset2['percentage_groei'].
 ↪max())
plt.show()
```



[36]:
```
plt.figure(figsize=(10, 6))
ax = sns.barplot(
    data=subset2.reset_index(),
    x='Stadsdeel',
    y='groei',
)

plt.title('groei leeftijdsgroep 0-15 jaar per wijk (2018-2021)')

ax.axhline(0, color='black')
ax.set_ylim(subset2['groei'].min(), subset2['groei'].max())
plt.show()
```

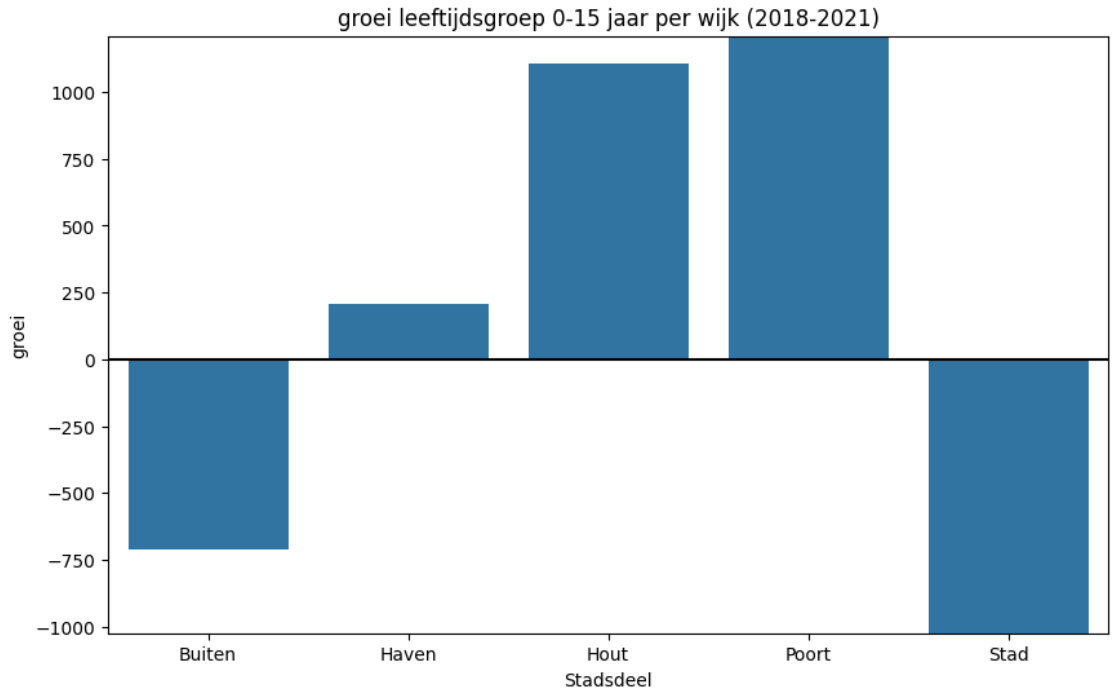


[ ]:

```

```

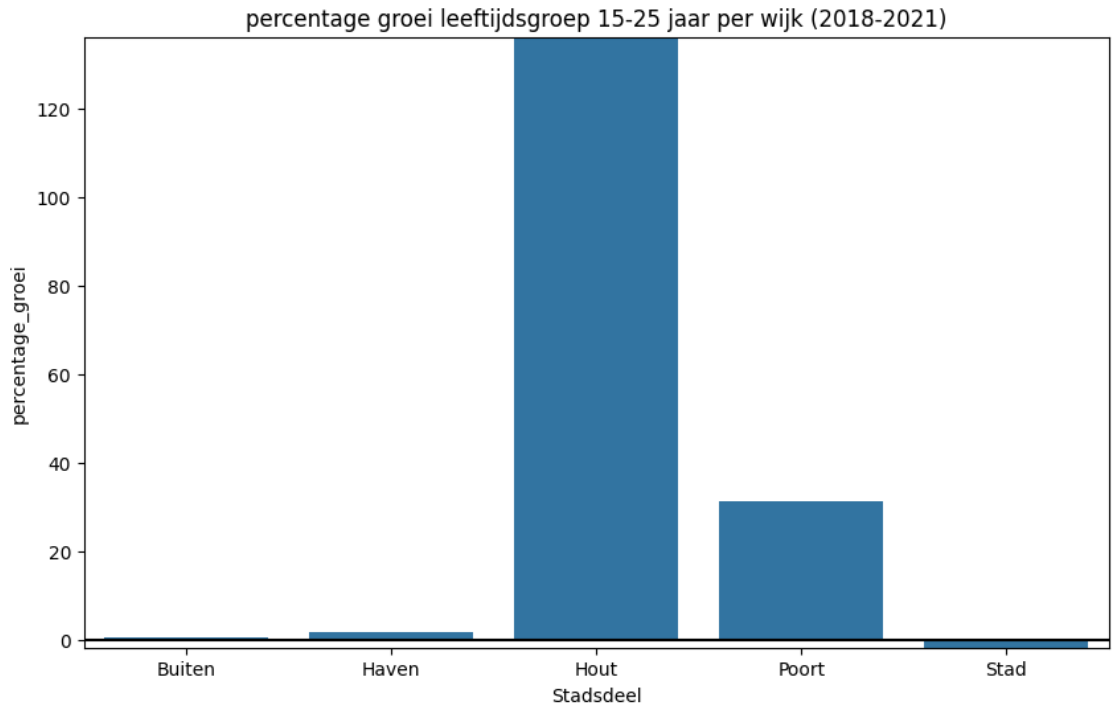
[37]:

```
plt.figure(figsize=(10, 6))
ax = sns.barplot(
    data=subset2.reset_index(),
    x='Stadsdeel',
    y='groei',
)

plt.title('groei leeftijdsgroep 0-15 jaar per wijk (2018-2021)')

ax.axhline(0, color='black')
ax.set_ylim(subset2['groei'].min(), subset2['groei'].max())
plt.show()
```

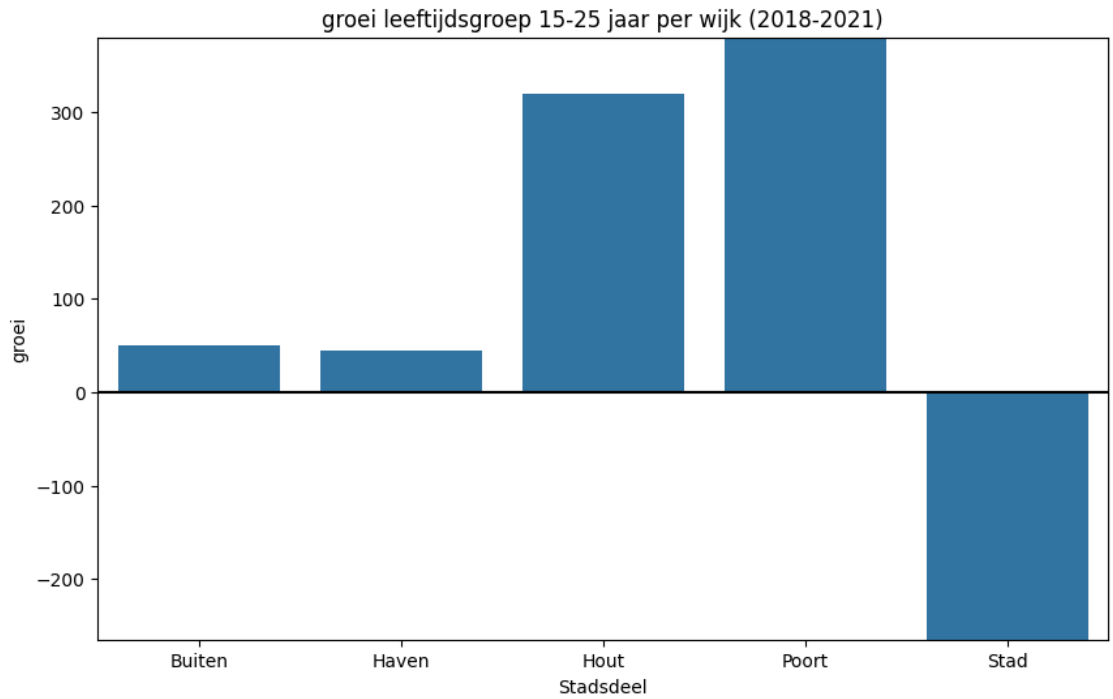


[38]:
```
subset3 = df_groei

subset3['Peildatum'] = pd.to_datetime(subset3['Peildatum'])

subset3 = subset3[subset3['Jaar'].isin([2018, 2021])]

subset3 = subset3.pivot(index='Stadsdeel', columns='Jaar',␣
 ↪values='Aantal_15_25')

subset3["groei"] = subset3[2021] - subset3[2018]

subset3["percentage_groei"] = (subset3["groei"] / subset3[2018]) * 100
plt.figure(figsize=(10, 6))
ax = sns.barplot(
    data=subset3.reset_index(),
    x='Stadsdeel',
    y='percentage_groei',
)

plt.title('percentage groei leeftijdsgroep 15-25 jaar per wijk (2018-2021)')

ax.axhline(0, color='black')
ax.set_ylim(subset3['percentage_groei'].min(), subset3['percentage_groei'].
 ↪max())
```

```
plt.show()
```



[39]:
```
plt.figure(figsize=(10, 6))
ax = sns.barplot(
    data=subset3.reset_index(),
    x='Stadsdeel',
    y='groei',
)

plt.title('groei leeftijdsgroep 15-25 jaar per wijk (2018-2021)')

ax.axhline(0, color='black')
ax.set_ylim(subset3['groei'].min(), subset3['groei'].max())
plt.show()
```



[40]:
```
df = pd.read_csv("../../data/processed/df_v8_ready_for_analysis.csv")
df.columns = df.columns.astype(str).str.strip()

# Peildatum -> Jaar
df["Peildatum"] = pd.to_datetime(df["Peildatum"], dayfirst=True,␣
↪errors="coerce")
df["Jaar"] = df["Peildatum"].dt.year

# Alleen consistente jaren
df = df[df["Jaar"] <= 2021].copy()

# wijkcodes opschonen
df["Wijk"] = df["Wijk"].astype(str).str.strip()

# Ratio's
df["Ratio_0_15"] = df["Aantal_0_15"] / df["AantalInwoners"]
df["Ratio_15_25"] = df["Aantal_15_25"] / df["AantalInwoners"]

# Checks
print("Min/Max Ratio 0-15:", df["Ratio_0_15"].min(), df["Ratio_0_15"].max())
print("Min/Max Ratio 15-25:", df["Ratio_15_25"].min(), df["Ratio_15_25"].max())
```

```
Min/Max Ratio 0-15: 0.17207303974221266 0.25566825775656327
Min/Max Ratio 15-25: 0.08333333333333333 0.13963489555906616
```

[41]:
```
subset4 = df_groei

subset4['Peildatum'] = pd.to_datetime(subset4['Peildatum'])

subset4 = subset4[subset4['Jaar'].isin([2018, 2021])]

subset4 = subset4.pivot(index='Stadsdeel', columns='Jaar',␣
 ↪values='AantalInwoners')

subset4["groei"] = subset4[2021] - subset4[2018]

subset4["percentage_groei"] = (subset4["groei"] / subset4[2018]) * 100
```

Dit onderdeel laat per stadsdeel zien hoe de leeftijdsgroepen en de totale bevolking zich tussen 2018 en 2021 hebben ontwikkeld.

Toelichting

De grafieken laten zien dat de ontwikkeling van jongeren verschilt per stadsdeel. In stadsdelen zoals Hout en Poort is een duidelijke groei te zien in het aantal kinderen en jongeren. In andere stadsdelen, zoals Stad en Buiten, is juist een lichte daling zichtbaar in de periode 2018–2021.

De grafieken met procentuele groei maken deze verschillen nog duidelijker. Zo laat Hout een sterke procentuele groei zien, terwijl sommige andere stadsdelen een negatieve groei hebben. Dit betekent dat de jeugdpopulatie zich niet overal in Almere op dezelfde manier ontwikkelt. Deze verschillen kunnen belangrijk zijn voor de planning van voorzieningen zoals scholen, sportfaciliteiten en jeugdzorg.

3.6 3.6 Correlatiematrix en multicollineariteit analyseren (Mohammed)

In deze stap wordt onderzocht hoe de verschillende numerieke variabelen met elkaar samenhangen. Dit helpt bij het herkennen van mogelijke multicollineariteit, waarbij meerdere variabelen vrijwel dezelfde informatie bevatten.

In deze stap analyseren we dus de samenhang tussen de verschillende numerieke variabelen in de dataset. Hiervoor berekenen we een correlatiematrix en visualiseren we deze met een heatmap. Dit helpt om te begrijpen welke variabelen sterk met elkaar samenhangen en welke variabelen mogelijk vergelijkbare informatie bevatten.

[42]:
```
df = pd.read_csv("../../data/processed/df_v10_ratio_temp.csv")

df.head()
```

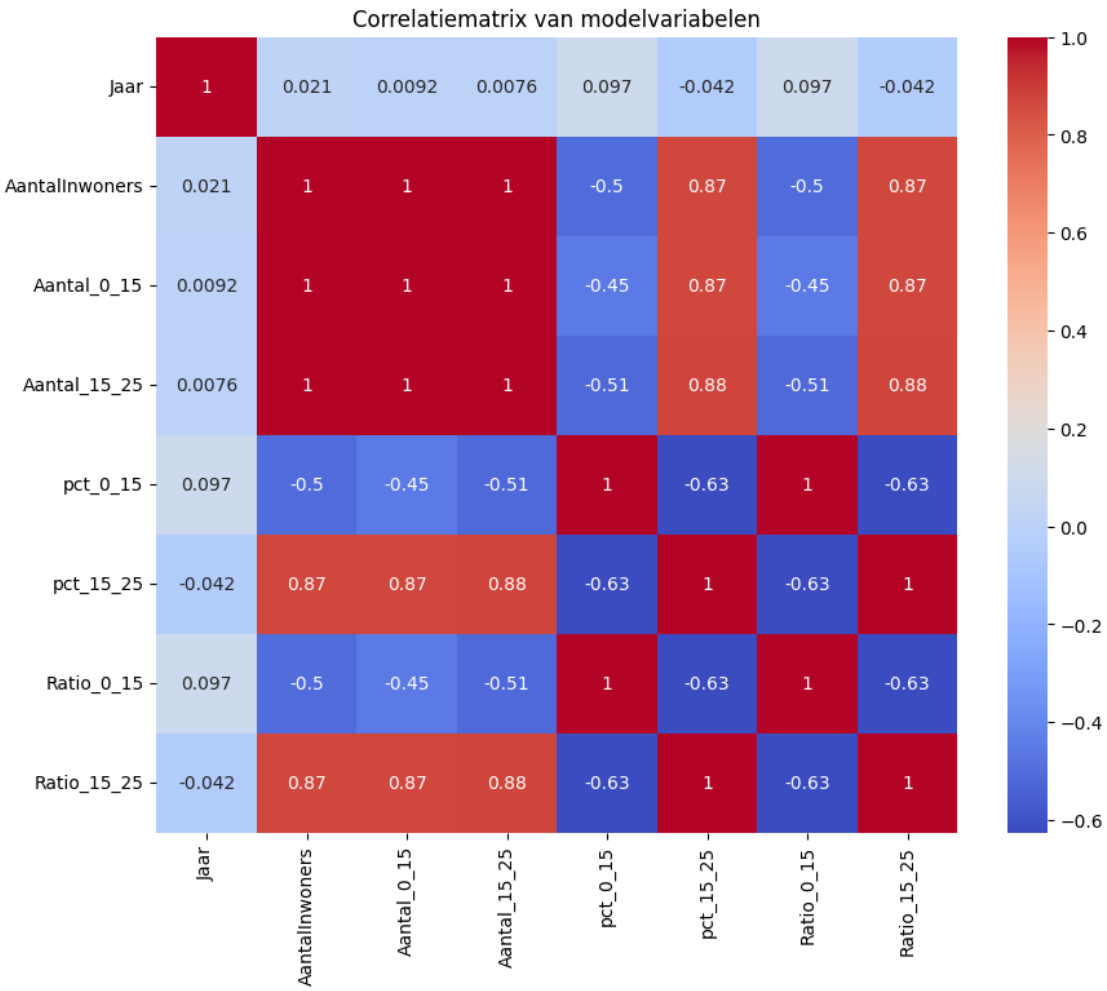
[42]:
```
    Peildatum  Jaar  WijkCode      Wijk  AantalInwoners  Aantal_0_15  \
0  2018-01-01  2018  WK003401  WK003401           22790         3975
1  2019-01-01  2019  WK003401  WK003401           22875         3945
2  2020-01-01  2020  WK003401  WK003401           23275         4005
3  2021-01-01  2021  WK003401  WK003401           23530         4180
4  2018-01-01  2018  WK003402  WK003402          108800        19915
```

```
   Aantal_15_25   pct_0_15  pct_15_25  Ratio_0_15  Ratio_15_25
0          2420  17.441860  10.618692    0.174419     0.106187
1          2415  17.245902  10.557377    0.172459     0.105574
2          2485  17.207304  10.676692    0.172073     0.106767
3          2465  17.764556  10.475988    0.177646     0.104760
4         14890  18.304228  13.685662    0.183042     0.136857
```

[43]:
```python
numeric_df = df.select_dtypes(include=["number"])
corr_matrix = numeric_df.corr()

corr_matrix
```

[43]:
```
                    Jaar  AantalInwoners  Aantal_0_15  Aantal_15_25  pct_0_15  \
Jaar            1.000000        0.021245     0.009155      0.007600  0.096953
AantalInwoners  0.021245        1.000000     0.996688      0.998529 -0.503183
Aantal_0_15     0.009155        0.996688     1.000000      0.995371 -0.447969
Aantal_15_25    0.007600        0.998529     0.995371      1.000000 -0.511022
pct_0_15        0.096953       -0.503183    -0.447969     -0.511022  1.000000
pct_15_25      -0.042248        0.867106     0.868357      0.879091 -0.625288
Ratio_0_15      0.096953       -0.503183    -0.447969     -0.511022  1.000000
Ratio_15_25    -0.042248        0.867106     0.868357      0.879091 -0.625288

                pct_15_25  Ratio_0_15  Ratio_15_25
Jaar            -0.042248    0.096953    -0.042248
AantalInwoners   0.867106   -0.503183     0.867106
Aantal_0_15      0.868357   -0.447969     0.868357
Aantal_15_25     0.879091   -0.511022     0.879091
pct_0_15        -0.625288    1.000000    -0.625288
pct_15_25        1.000000   -0.625288     1.000000
Ratio_0_15      -0.625288    1.000000    -0.625288
Ratio_15_25      1.000000   -0.625288     1.000000
```

[44]:
```python
import seaborn as sns
import matplotlib.pyplot as plt

plt.figure(figsize=(10,8))
sns.heatmap(corr_matrix, annot=True, cmap="coolwarm")

plt.title("Correlatiematrix van modelvariabelen")
plt.show()
```



De correlatiematrix laat zien welke variabelen sterk samenhangen. Dit is belangrijk om te beoordelen welke variabelen later geschikt zijn voor gebruik in het model.

Toelichting

De correlatiematrix laat zien dat het totaal aantal inwoners, het aantal kinderen (0–15 jaar) en het aantal jongeren (15–25 jaar) zeer sterk positief met elkaar samenhangen. Dit betekent dat wijken met meer inwoners doorgaans ook meer jongeren hebben.

Daarnaast is te zien dat de percentage- en ratio-variabelen sterk met elkaar correleren, omdat deze dezelfde informatie weergeven maar op een andere manier zijn berekend. Dit kan belangrijk zijn bij het bouwen van een model, omdat variabelen met een zeer hoge correlatie mogelijk dezelfde informatie bevatten. Het is daarom verstandig om niet alle sterk gecorreleerde variabelen tegelijk in een model te gebruiken.

3.7 3.7 Noah Dataset Finaliseren

Voor modellering mogen meestal alleen numerieke variabelen gebruikt worden.

Niet-numerieke kolommen zoals:

Wijknamen (tekst)

Datums

Eventuele categorieën

kunnen een model verstoren of errors geven.

Daarom:

1 We behouden alleen numerieke kolommen 2 We verwijderen overbodige variabelen 3 We controleren nog een keer op missende waarden 4 We maken een definitieve dataset

[45]:
```
#controle bestand inleezen en kijken of alle waardes numeriek zijn
import pandas as pd
df = pd.read_csv("../../data/processed/df_v10_ratio_temp.csv")
df.head()

df.dtypes
```

[45]:
```
Peildatum          object
Jaar                int64
WijkCode           object
Wijk               object
AantalInwoners      int64
Aantal_0_15         int64
Aantal_15_25        int64
pct_0_15          float64
pct_15_25         float64
Ratio_0_15        float64
Ratio_15_25       float64
dtype: object
```

[46]:
```
# Peildatum omzetten naar datetime zodat we peildatum numeriek kunnen maken
df['Peildatum'] = pd.to_datetime(df['Peildatum'])

# Jaar extraheren
df['jaar'] = df['Peildatum'].dt.year
```

Nu hebben we van peildatum "jaar" gemaakt met een numerieke waarden nu kunnen we de oude verie deleten oftewel peildatm en dit gaan we gelijk controleren

[47]:
```
df = df.drop(columns=['Peildatum'])
#controleren of peildatum weg is en de numerieke vervaning jaar ook␣
↪daadwerkelijk numeriek is
df.dtypes
```

[47]:
```
Jaar                int64
WijkCode           object
```

```
Wijk                object
AantalInwoners       int64
Aantal_0_15          int64
Aantal_15_25         int64
pct_0_15           float64
pct_15_25          float64
Ratio_0_15         float64
Ratio_15_25        float64
jaar                 int32
dtype: object
```

[48]:
```
# isna() markeert elke cel als True (bij NaN) of False, en sum() telt␣
↪vervolgens alle True waardes op per kolom. zo controleren we of er nog NaN␣
↪waardes zijn bij laatste dataset
df.isna().sum()
```

[48]:
```
Jaar              0
WijkCode          0
Wijk              0
AantalInwoners    0
Aantal_0_15       0
Aantal_15_25      0
pct_0_15          0
pct_15_25         0
Ratio_0_15        0
Ratio_15_25       0
jaar              0
dtype: int64
```

3.8 3.8 Features samenvoegen tot één modeldataset (Faysal)

In deze stap worden de beschikbare feature-datasets samengevoegd tot één modelklare dataset. De merge gebeurt op de sleutel `Wijk` en `Jaar`.

Samenvoegen van feature-datasets voor modellering

In deze stap worden de datasets met basisvariabelen en ratio-variabelen samengevoegd tot één dataset die gebruikt kan worden voor modellering. De basisdataset bevat per wijk en per jaar het aantal inwoners en het aantal jongeren. De ratio-dataset bevat aanvullende variabelen die het aandeel jongeren binnen de bevolking weergeven. De samenvoeging gebeurt op basis van de combinatie van wijk en jaar.

[49]:
```
import pandas as pd
from pathlib import Path

base_path = Path("../../data/processed/df_v8_ready_for_analysis.csv")
df_base = pd.read_csv(base_path)
```

```
df_base.head()
```

[49]:

```
    Peildatum  Jaar  WijkCode        Wijk  AantalInwoners  Aantal_0_15  \
0  2018-01-01  2018  WK003401  WK003401             22790         3975
1  2019-01-01  2019  WK003401  WK003401             22875         3945
2  2020-01-01  2020  WK003401  WK003401             23275         4005
3  2021-01-01  2021  WK003401  WK003401             23530         4180
4  2022-01-01  2022  WK003401  WK003401              1635          175

   Aantal_15_25   pct_0_15  pct_15_25
0          2420  17.441860  10.618692
1          2415  17.245902  10.557377
2          2485  17.207304  10.676692
3          2465  17.764556  10.475988
4           150  10.703364   9.174312
```

[50]:

```
df_base[["Wijk","Jaar"]].head()

df_base.duplicated(subset=["Wijk","Jaar"]).sum()
```

[50]: 0

[51]:

```
ratio_path = Path("../../data/processed/df_v10_ratio_temp.csv")

df_ratio = pd.read_csv(ratio_path)

df_ratio.head()
```

[51]:

```
    Peildatum  Jaar  WijkCode      Wijk  AantalInwoners  Aantal_0_15  \
0  2018-01-01  2018  WK003401  WK003401           22790         3975
1  2019-01-01  2019  WK003401  WK003401           22875         3945
2  2020-01-01  2020  WK003401  WK003401           23275         4005
3  2021-01-01  2021  WK003401  WK003401           23530         4180
4  2018-01-01  2018  WK003402  WK003402          108800        19915

   Aantal_15_25   pct_0_15  pct_15_25  Ratio_0_15  Ratio_15_25
0          2420  17.441860  10.618692    0.174419     0.106187
1          2415  17.245902  10.557377    0.172459     0.105574
2          2485  17.207304  10.676692    0.172073     0.106767
3          2465  17.764556  10.475988    0.177646     0.104760
4         14890  18.304228  13.685662    0.183042     0.136857
```

[52]:

```
df_ratio = df_ratio[[
    "Wijk",
    "Jaar",
    "Ratio_0_15",
    "Ratio_15_25"
```

```
]]

df_ratio.head()
```

[52]:

	Wijk	Jaar	Ratio_0_15	Ratio_15_25
0	WK003401	2018	0.174419	0.106187
1	WK003401	2019	0.172459	0.105574
2	WK003401	2020	0.172073	0.106767
3	WK003401	2021	0.177646	0.104760
4	WK003402	2018	0.183042	0.136857

[53]:

```
df_base["Wijk"] = df_base["Wijk"].astype(str).str.strip()
df_ratio["Wijk"] = df_ratio["Wijk"].astype(str).str.strip()

df_base["Jaar"] = df_base["Jaar"].astype(int)
df_ratio["Jaar"] = df_ratio["Jaar"].astype(int)
df_merged = df_base.merge(
    df_ratio,
    on=["Wijk","Jaar"],
    how="left"
)

df_merged.head()
```

[53]:

	Peildatum	Jaar	WijkCode	Wijk	AantalInwoners	Aantal_0_15	\
0	2018-01-01	2018	WK003401	WK003401	22790	3975	
1	2019-01-01	2019	WK003401	WK003401	22875	3945	
2	2020-01-01	2020	WK003401	WK003401	23275	4005	
3	2021-01-01	2021	WK003401	WK003401	23530	4180	
4	2022-01-01	2022	WK003401	WK003401	1635	175	

	Aantal_15_25	pct_0_15	pct_15_25	Ratio_0_15	Ratio_15_25
0	2420	17.441860	10.618692	0.174419	0.106187
1	2415	17.245902	10.557377	0.172459	0.105574
2	2485	17.207304	10.676692	0.172073	0.106767
3	2465	17.764556	10.475988	0.177646	0.104760
4	150	10.703364	9.174312	NaN	NaN

[54]:

```
print("Rows base:", len(df_base))
print("Rows merged:", len(df_merged))

df_merged.isnull().sum()
```

```
Rows base: 237
Rows merged: 237
```

[54]:

```
Peildatum           0
Jaar                0
```

```
WijkCode            0
Wijk                0
AantalInwoners      0
Aantal_0_15         0
Aantal_15_25        0
pct_0_15            0
pct_15_25           0
Ratio_0_15        217
Ratio_15_25       217
dtype: int64
```

[55]:
```
df_base.dtypes
df_ratio.dtypes
```

[55]:
```
Wijk            object
Jaar             int64
Ratio_0_15     float64
Ratio_15_25    float64
dtype: object
```

[56]:
```
df_ratio = df_ratio[df_ratio["Jaar"].isin(df_base["Jaar"].unique())].copy()

print("Jaren base:", sorted(df_base["Jaar"].unique()))
print("Jaren ratio (na filter):", sorted(df_ratio["Jaar"].unique()))
```

```
Jaren base: [2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025]
Jaren ratio (na filter): [2018, 2019, 2020, 2021]
```

[57]:
```
import pandas as pd
from pathlib import Path

lag_path = Path("../../data/processed/df_v12_model_ready.csv")
df_lag = pd.read_csv(lag_path)

ratio_path = Path("../../data/processed/df_v10_ratio_temp.csv")
df_ratio = pd.read_csv(ratio_path)

df_ratio = df_ratio[["Wijk", "Jaar", "Ratio_0_15", "Ratio_15_25"]].copy()

# Wijk opschonen
df_lag["Wijk"] = df_lag["Wijk"].astype(str).str.strip().str.upper()
df_ratio["Wijk"] = df_ratio["Wijk"].astype(str).str.strip().str.upper()

# Jaar opschonen
df_lag["Jaar"] = pd.to_numeric(df_lag["Jaar"], errors="coerce").astype("Int64")
df_ratio["Jaar"] = pd.to_numeric(df_ratio["Jaar"], errors="coerce").
 ↪astype("Int64")
```

```python
# Merge op Wijk + Jaar
df_final = df_lag.merge(df_ratio, on=["Wijk", "Jaar"], how="left")

# Check hoeveel matches
matches = df_final[["Ratio_0_15", "Ratio_15_25"]].notna().all(axis=1).sum()
totaal = len(df_final)

print("Aantal rijen met ratio's gevuld:", matches, "van", totaal)
print("Aantal rijen zonder ratio's:", totaal - matches)

# Opslaan
output_path = Path("../../data/processed/df_v12_model_ready.csv")
df_final.to_csv(output_path, index=False)

df_final.head()
```

```
Aantal rijen met ratio's gevuld: 15 van 181
Aantal rijen zonder ratio's: 166
```

[57]:

```
     Peildatum  Jaar  WijkCode      Wijk  AantalInwoners  Aantal_0_15  \
0  2019-01-01  2019  WK003401  WK003401           22875         3945
1  2020-01-01  2020  WK003401  WK003401           23275         4005
2  2021-01-01  2021  WK003401  WK003401           23530         4180
3  2022-01-01  2022  WK003401  WK003401            1635          175
4  2023-01-01  2023  WK003401  WK003401            1740          190

   Aantal_15_25   pct_0_15  pct_15_25  Jeugdzorg  Jeugdzorg_lag1  Ratio_0_15  \
0          2415  17.245902  10.557377       6360          6395.0    0.172459
1          2485  17.207304  10.676692       6490          6360.0    0.172073
2          2465  17.764556  10.475988       6645          6490.0    0.177646
3           150  10.703364   9.174312        325          6645.0         NaN
4           160  10.919540   9.195402        350           325.0         NaN

   Ratio_15_25
0     0.105574
1     0.106767
2     0.104760
3          NaN
4          NaN
```

[58]:

```python
print("df_lag:")
print(df_lag.columns.tolist())

print("\ndf_ratio:")
print(df_ratio.columns.tolist())
```

```
df_lag:
['Peildatum', 'Jaar', 'WijkCode', 'Wijk', 'AantalInwoners', 'Aantal_0_15',
```

```
'Aantal_15_25', 'pct_0_15', 'pct_15_25', 'Jeugdzorg', 'Jeugdzorg_lag1']

df_ratio:
['Wijk', 'Jaar', 'Ratio_0_15', 'Ratio_15_25']
```

[59]:
```
matches = df_merged[["Ratio_0_15", "Ratio_15_25"]].notna().all(axis=1).sum()
totaal = len(df_merged)

print("Aantal rijen met ratio's gevuld:", matches, "van", totaal)
print("Aantal rijen zonder ratio's:", totaal - matches)
```

```
Aantal rijen met ratio's gevuld: 20 van 237
Aantal rijen zonder ratio's: 217
```

[60]:
```
df_final.dtypes
```

[60]:
```
Peildatum          object
Jaar                Int64
WijkCode           object
Wijk               object
AantalInwoners      int64
Aantal_0_15         int64
Aantal_15_25        int64
pct_0_15          float64
pct_15_25         float64
Jeugdzorg           int64
Jeugdzorg_lag1    float64
Ratio_0_15        float64
Ratio_15_25       float64
dtype: object
```

[61]:
```
# output_path = Path("../../data/processed/df_v12_model_ready.csv")

# output_path
```

[62]:
```
# df_model.groupby("Wijk")["Jaar"].min(), df_model.groupby("Wijk")["Jaar"].max()
```

[63]:
```
df_ratio["Jaar"].unique()
df_base["Jaar"].unique()
```

[63]:
```
array([2018, 2019, 2020, 2021, 2022, 2023, 2024, 2025])
```

Toelichting

Na het samenvoegen van de datasets ontstaat één modeldataset met de belangrijkste variabelen voor verdere analyse. In totaal bevat de dataset 237 rijen. Van deze rijen hebben er 20 ook de berekende ratio-variabelen, omdat deze alleen beschikbaar zijn voor de jaren 2018 tot en met 2021. Voor de jaren daarna ontbreken deze waarden.

De uiteindelijke dataset bevat per wijk en per jaar het totaal aantal inwoners, het aantal inwon-

ers van 0–15 jaar, het aantal inwoners van 15–25 jaar en de bijbehorende ratio's. Deze dataset wordt opgeslagen als `df_v12_model_ready.csv` en vormt de basis voor de volgende fase waarin het voorspellende model wordt gebouwd.

4 Fase 4-Modeling

In deze fase wordt het regressiemodel ontwikkeld en geëvalueerd. De model-klare dataset uit fase 3 wordt gebruikt om een voorspellend model te trainen dat de ontwikkeling van de jeugdpopulatie per wijk kan schatten.

De input van deze fase is:

`../../data/processed/df_v12_model_ready.csv`

Het doel van deze fase is om op basis van de voorbereide dataset een regressiemodel te bouwen en de prestaties van het model te evalueren. Binnen deze fase zijn door verschillende teamleden verschillende modellen getest en met elkaar vergeleken.

De onderdelen die in deze fase worden uitgevoerd zijn onder andere: - het selecteren van features en de targetvariabele; - het splitsen van de dataset in een train- en testset; - het trainen van één of meerdere regressiemodellen; - het evalueren van de modelprestaties aan de hand van evaluatiematen.

De uiteindelijke output van deze fase bestaat uit modelresultaten, evaluatiematen en visualisaties die inzicht geven in de prestaties van het model.

[64]:
```
# Basis dataset inladen
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_absolute_error, mean_squared_error

df = pd.read_csv("../../data/processed/df_v12_model_ready.csv")

print(df.columns.tolist())
df.head()
```

```
['Peildatum', 'Jaar', 'WijkCode', 'Wijk', 'AantalInwoners', 'Aantal_0_15',
'Aantal_15_25', 'pct_0_15', 'pct_15_25', 'Jeugdzorg', 'Jeugdzorg_lag1',
'Ratio_0_15', 'Ratio_15_25']
```

[64]:

	Peildatum	Jaar	WijkCode	Wijk	AantalInwoners	Aantal_0_15	\
0	2019-01-01	2019	WK003401	WK003401	22875	3945	
1	2020-01-01	2020	WK003401	WK003401	23275	4005	
2	2021-01-01	2021	WK003401	WK003401	23530	4180	
3	2022-01-01	2022	WK003401	WK003401	1635	175	
4	2023-01-01	2023	WK003401	WK003401	1740	190	

```
   Aantal_15_25   pct_0_15  pct_15_25  Jeugdzorg  Jeugdzorg_lag1  Ratio_0_15  \
0          2415  17.245902  10.557377       6360          6395.0    0.172459
1          2485  17.207304  10.676692       6490          6360.0    0.172073
2          2465  17.764556  10.475988       6645          6490.0    0.177646
3           150  10.703364   9.174312        325          6645.0         NaN
4           160  10.919540   9.195402        350           325.0         NaN

   Ratio_15_25
0     0.105574
1     0.106767
2     0.104760
3          NaN
4          NaN
```

5 4.1 Lag Model (Mohammed-Wail)

In dit model wordt onderzocht of de historische waarde van de jeugdpopulatie een goede voorspeller is voor de huidige jeugdpopulatie in een wijk. Hiervoor wordt gebruikgemaakt van de lag-feature Jeugdzorg_lag1, die in fase 3 is aangemaakt. Deze variabele bevat de totale jeugdpopulatie van het voorgaande jaar binnen dezelfde wijk.

Het idee achter dit model is dat bevolkingsontwikkelingen vaak geleidelijk verlopen. Wanneer een wijk in een bepaald jaar een grote jeugdpopulatie heeft, is de kans groot dat deze populatie in het volgende jaar nog steeds relatief hoog blijft.

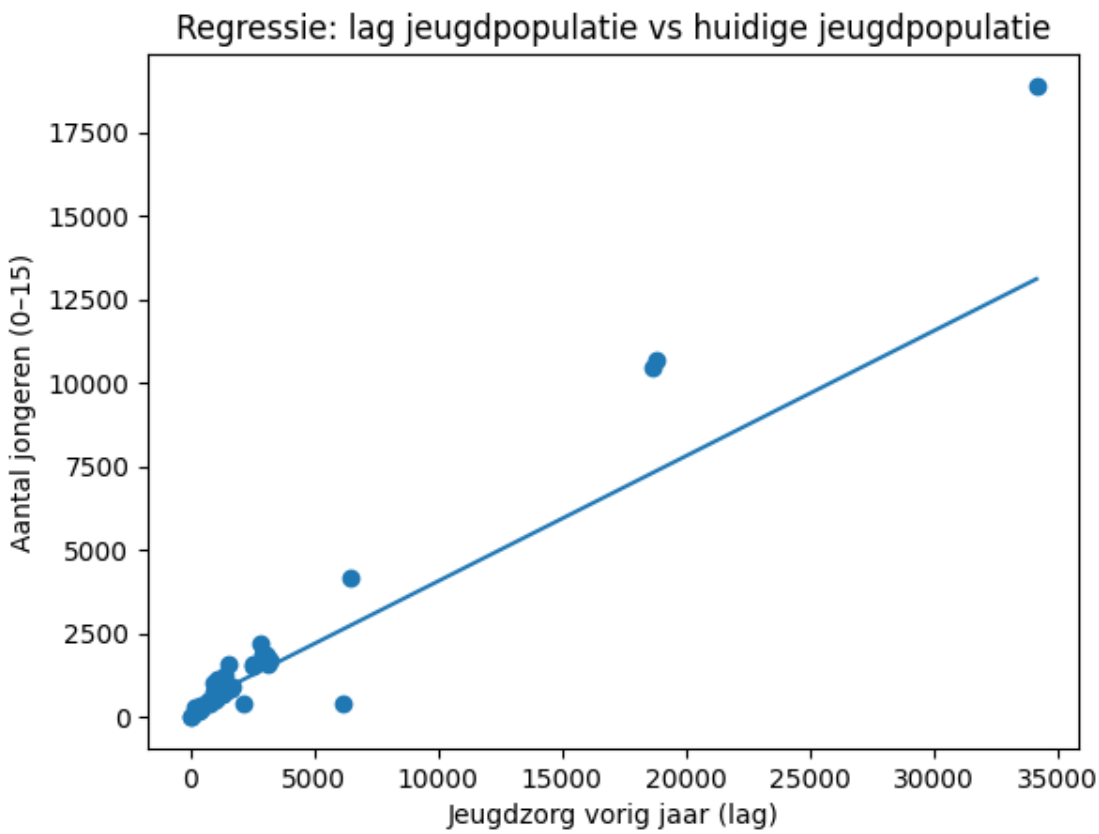
Door middel van een enkelvoudige lineaire regressie wordt onderzocht of deze historische waarde een sterke relatie heeft met het huidige aantal jongeren van 0–15 jaar.

[76]:
```
df_lag_model = df[["Jeugdzorg_lag1", "Aantal_0_15"]].dropna()

X = df_lag_model[["Jeugdzorg_lag1"]]
y = df_lag_model["Aantal_0_15"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

r2_lag = r2_score(y_test, y_pred)

print("R²:", r2_lag)
```

```
R²: 0.8725717947880063
```

[66]:
```python
import matplotlib.pyplot as plt
import numpy as np

plt.scatter(X_test, y_test)

x_line = np.linspace(X_test.min(), X_test.max(), 100)
y_line = model.predict(x_line)

plt.plot(x_line, y_line)

plt.xlabel("Jeugdzorg vorig jaar (lag)")
plt.ylabel("Aantal jongeren (0-15)")
plt.title("Regressie: lag jeugdpopulatie vs huidige jeugdpopulatie")

plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2691:
UserWarning: X does not have valid feature names, but LinearRegression was
fitted with feature names
  warnings.warn(
```



Het lag-model toont in welke mate de jeugdpopulatie van het voorgaande jaar de huidige populatie verklaart. De R^2-waarde laat zien hoe goed de historische waarde als voorspeller werkt. Dit model geeft ons een eerste indruk of stabiliteit in de tijd een rol speelt in de wijkpopulatie.

6 4.2 - Ratio Model

Nu onderzoeken we of de verhouding van jongeren ten opzichte van de totale bevolking in een wijk een voorspellende waarde heeft. We gebruiken de ratio (jongeren/totaal) als predictor om te zien of relatieve samenstelling iets zegt over het absolute aantal jongeren

[67]:
```
import statsmodels.api as sm
from sklearn.model_selection import train_test_split
import numpy as np

df_ratio_model = df[["Ratio_0_15", "Aantal_0_15"]].replace([np.inf, -np.inf],␣
 ↪np.nan).dropna()

X = df_ratio_model[["Ratio_0_15"]]
y = df_ratio_model["Aantal_0_15"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

X_train_ols = sm.add_constant(X_train)
X_test_ols = sm.add_constant(X_test)

model_ratio = sm.OLS(y_train, X_train_ols).fit()

print(model_ratio.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:            Aantal_0_15   R-squared:                       0.272
Model:                            OLS   Adj. R-squared:                  0.181
Method:                 Least Squares   F-statistic:                     2.988
Date:                Sat, 07 Mar 2026   Prob (F-statistic):              0.122
Time:                        15:39:57   Log-Likelihood:                -100.33
No. Observations:                  10   AIC:                             204.7
Df Residuals:                       8   BIC:                             205.3
Df Model:                           1
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const       3.336e+04   1.45e+04      2.305      0.050     -10.598    6.67e+04
Ratio_0_15 -1.259e+05   7.28e+04     -1.729      0.122   -2.94e+05    4.21e+04
==============================================================================
```

```
Omnibus:                        0.702   Durbin-Watson:                   1.970
Prob(Omnibus):                  0.704   Jarque-Bera (JB):                0.562
Skew:                           0.097   Prob(JB):                        0.755
Kurtosis:                       1.855   Cond. No.                         38.9
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly
specified.
```

Het ratio-model toont hoe sterk de verhouding tussen jongeren en de totale bevolking samenhangt met het absolute aantal jongeren van 0–15 jaar. Afhankelijk van de R^2 in de samenvatting zie je of de ratio een significante voorspeller is. Bij een hoge R^2 betekent dit dat de relatieve samenstelling van de wijkbevolking veel zegt over het absolute aantal jongeren

7 4.3 - Groei Model

In dit model wordt onderzocht of de procentuele groei van de jeugdpopulatie een bruikbare voorspeller is voor het huidige aantal jongeren van 0–15 jaar. Hiervoor wordt de variabele Groei_Jeugd_pct gebruikt.

[77]:
```
# groei-feature maken
df = df.sort_values(["Wijk", "Jaar"]).copy()
df["Groei_Jeugd_pct"] = df.groupby("Wijk")["Aantal_0_15"].pct_change() * 100

df_model = df[["Groei_Jeugd_pct", "Aantal_0_15"]].dropna()

X = df_model[["Groei_Jeugd_pct"]]
y = df_model["Aantal_0_15"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

r2_groei = r2_score(y_test, y_pred)

print("R²:", r2_groei)
```

```
R²: -0.004547242103004168
```

Het groeimodel onderzoekt of de procentuele verandering van de jeugdpopulatie een bruikbare voorspeller is voor het huidige aantal jongeren van 0–15 jaar. basis van de R^2-waarde kan worden beoordeeld in hoeverre deze groeivariabele het aantal jongeren kan verklaren. Wanneer de R^2 relatief laag is, betekent dit dat de procentuele groei van de jeugdpopulatie slechts een beperkte

invloed heeft op het absolute aantal jongeren in een wijk.

Dit suggereert dat andere factoren, zoals de totale bevolkingsomvang of historische populatieaantallen, mogelijk een sterkere voorspeller zijn voor de omvang van de jeugdpopulatie.

8 4.4 - Aantal inwoners Model

In dit model wordt onderzocht of het totale aantal inwoners van een wijk een goede voorspeller is voor het aantal jongeren van 0–15 jaar.

De gedachte achter dit model is dat grotere wijken doorgaans ook een grotere jeugdpopulatie hebben. Door middel van een lineaire regressie wordt onderzocht of er een duidelijke relatie bestaat tussen het totale aantal inwoners en het aantal jongeren.

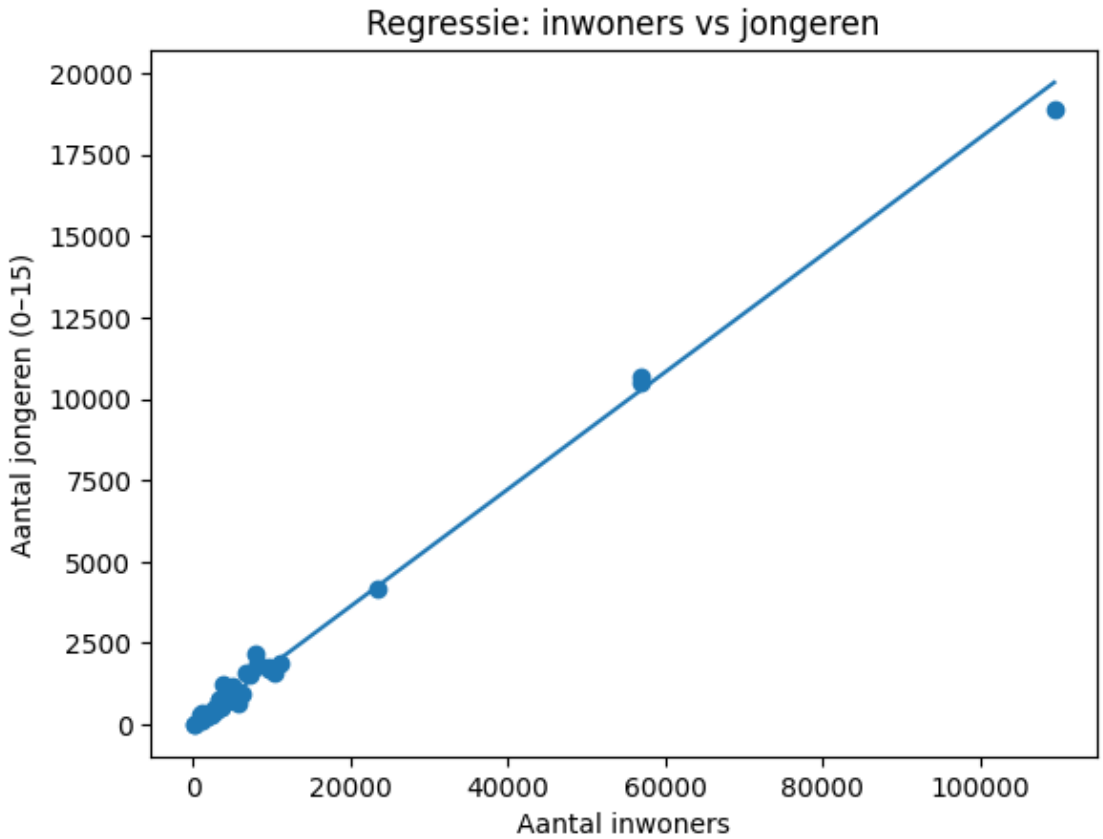
[78]:
```
df_model = df[["AantalInwoners", "Aantal_0_15"]].dropna()

X = df_model[["AantalInwoners"]]
y = df_model["Aantal_0_15"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

r2_inwoners = r2_score(y_test, y_pred)

print("R²:", r2_inwoners)
```

```
R²: 0.9944004056530807
```

[70]:
```
plt.scatter(X_test, y_test)

x_line = np.linspace(X_test.min(), X_test.max(), 100)
y_line = model.predict(x_line)

plt.plot(x_line, y_line)

plt.xlabel("Aantal inwoners")
plt.ylabel("Aantal jongeren (0-15)")
plt.title("Regressie: inwoners vs jongeren")

plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2691:
UserWarning: X does not have valid feature names, but LinearRegression was
```

```
fitted with feature names
  warnings.warn(
```



Het inwonersmodel onderzoekt of de totale bevolkingsomvang van een wijk samenhangt met het aantal jongeren van 0–15 jaar. De R^2-waarde laat zien in hoeverre het aantal inwoners de variatie in de jeugdpopulatie kan verklaren.

Wanneer de R^2 relatief hoog is, suggereert dit dat grotere wijken ook meer jongeren bevatten. Dit betekent dat de totale bevolkingsomvang een bruikbare indicator kan zijn bij het inschatten van de jeugdpopulatie per wijk.

9 4.5 Jeugdzorg Model

In dit model wordt onderzocht of de variabele `Jeugdzorg` een bruikbare voorspeller is voor het aantal jongeren van 0–15 jaar.

De variabele `Jeugdzorg` is in fase 3 samengesteld als de som van de leeftijdsgroepen 0–15 jaar en 15–25 jaar. Met dit model wordt bekeken of deze bredere jeugdvariabele samenhang vertoont met de jongere doelgroep binnen de wijk. Code

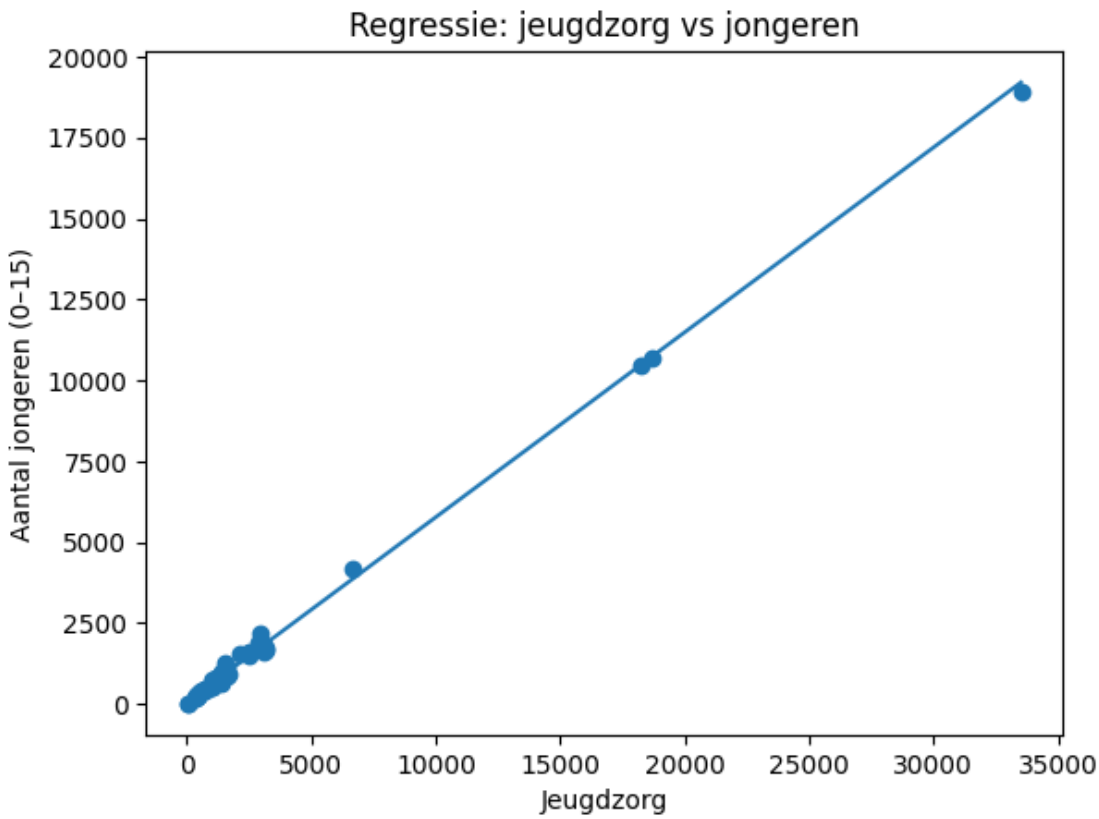
[71]:
```
df["Jeugdzorg"] = df["Aantal_0_15"] + df["Aantal_15_25"]
```

[79]:
```
df_model = df[["Jeugdzorg", "Aantal_0_15"]].dropna()

X = df_model[["Jeugdzorg"]]
y = df_model["Aantal_0_15"]

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.3, random_state=42
)

model = LinearRegression()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

r2_jeugdzorg = r2_score(y_test, y_pred)

print("R²:", r2_jeugdzorg)
```

```
R²: 0.9979860398789951
```

[73]:
```
plt.scatter(X_test, y_test)

x_line = np.linspace(X_test.min(), X_test.max(), 100)
y_line = model.predict(x_line)

plt.plot(x_line, y_line)

plt.xlabel("Jeugdzorg")
plt.ylabel("Aantal jongeren (0-15)")
plt.title("Regressie: jeugdzorg vs jongeren")

plt.show()
```

```
/usr/local/lib/python3.12/dist-packages/sklearn/utils/validation.py:2691:
UserWarning: X does not have valid feature names, but LinearRegression was
fitted with feature names
  warnings.warn(
```



9.1 Evaluatie

In deze evaluatie worden de verschillende regressiemodellen met elkaar vergeleken. Hiervoor wordt de R^2-waarde gebruikt, die aangeeft hoeveel variatie in het aantal jongeren (0–15 jaar) door het model wordt verklaard. Door de R^2-waarden van de modellen naast elkaar te zetten, kunnen we bepalen welke variabele de sterkste relatie heeft met de jeugdpopulatie.

[80]:
```
resultaten = pd.DataFrame({
    "Model": ["Lag", "Ratio", "Groei", "AantalInwoners", "Jeugdzorg"],
    "R2": [r2_lag, model_ratio.rsquared, r2_groei, r2_inwoners, r2_jeugdzorg]
})

resultaten = resultaten.sort_values("R2", ascending=False)
resultaten
```

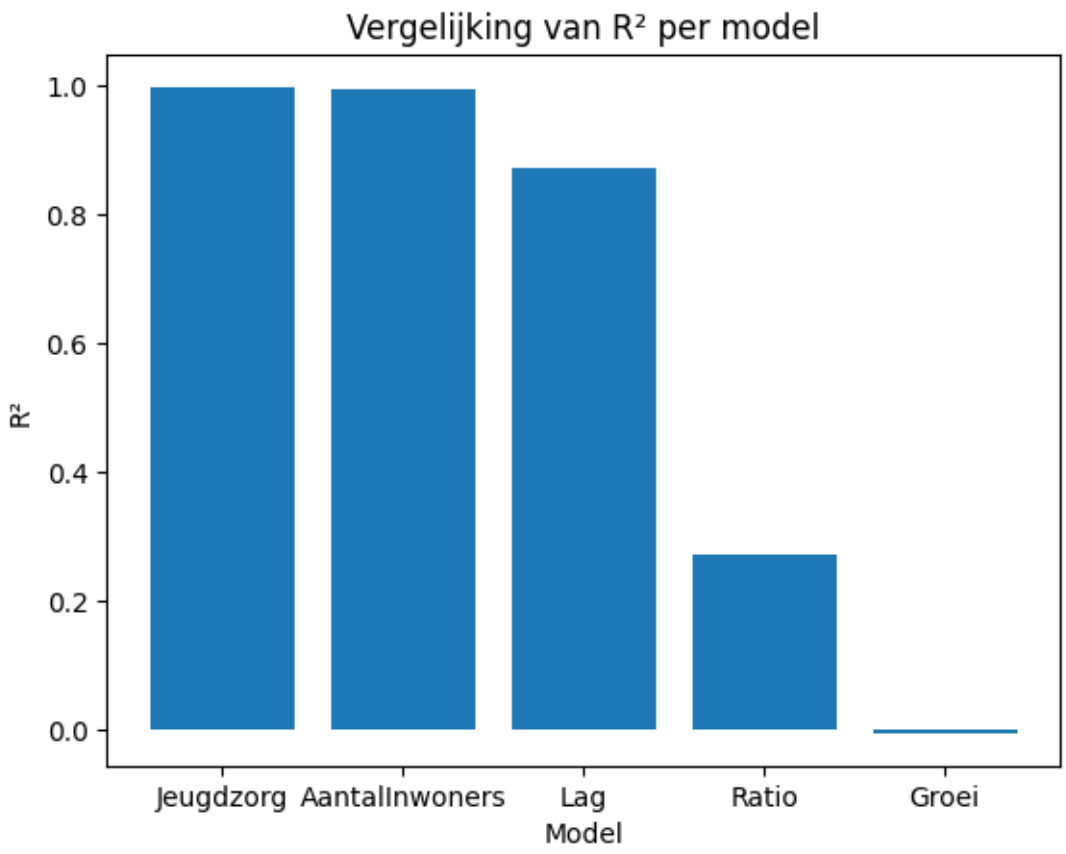
[80]:

	Model	R2
4	Jeugdzorg	0.997986
3	AantalInwoners	0.994400
0	Lag	0.872572
1	Ratio	0.271940
2	Groei	-0.004547

Uit de vergelijking van de modellen blijkt dat sommige variabelen beter in staat zijn om het aantal jongeren te verklaren dan andere. Het model met de hoogste R^2-waarde verklaart het grootste deel van de variatie in de data. Dit model kan daarom als de beste indicator worden gezien binnen deze analyse, al moeten de resultaten voorzichtig worden geïnterpreteerd vanwege de beperkte dataset.

9.2 Visualisatie Resultaten

[81]:
```
import matplotlib.pyplot as plt

plt.bar(resultaten["Model"], resultaten["R2"])
plt.title("Vergelijking van R² per model")
plt.ylabel("R²")
plt.xlabel("Model")
plt.show()
```



De visualisatie laat zien dat het jeugdzorgmodel en het aantal-inwonersmodel de hoogste R^2-waarden hebben. Dit betekent dat deze modellen de variatie in het aantal jongeren het beste kunnen verklaren.

9.3 Conclusie

Uit de verschillende regressiemodellen blijkt dat sommige variabelen het aantal jongeren (0–15 jaar) beter kunnen verklaren dan andere. Met name het jeugdzorgmodel en het aantal-inwonersmodel behalen zeer hoge R^2-waarden, wat betekent dat deze modellen een groot deel van de variatie in het aantal jongeren kunnen verklaren. Het lag-model laat ook een redelijk sterke relatie zien, wat aangeeft dat eerdere aantallen jongeren een goede indicatie kunnen geven voor toekomstige aantallen. Het ratio- en groeimodel presteren daarentegen duidelijk minder goed.

Op basis van deze resultaten kan worden gesteld dat het model gedeeltelijk bruikbaar kan zijn voor de gemeente. Het kan bijvoorbeeld helpen om inzicht te krijgen in welke wijken mogelijk een hogere jeugdpopulatie hebben. Deze informatie kan relevant zijn bij het plannen van voorzieningen zoals scholen, sportfaciliteiten of jeugdbeleid.

Er zijn echter ook beperkingen in deze analyse. De dataset bevat een relatief beperkt aantal jaren en observaties, waardoor de modellen minder robuust kunnen zijn. Daarnaast zijn sommige variabelen sterk aan elkaar gerelateerd, wat de interpretatie van de resultaten kan beïnvloeden. Ook blijkt dat niet alle variabelen even geschikt zijn om de jeugdpopulatie te voorspellen.

Voor vervolgonderzoek zou het nuttig zijn om meer jaren aan data toe te voegen en extra variabelen te onderzoeken die mogelijk invloed hebben op de ontwikkeling van de jeugdpopulatie. Daarnaast kan het interessant zijn om andere modeltypen te testen of meer geavanceerde methoden te gebruiken om te onderzoeken of de voorspellingen verder verbeterd kunnen worden.